

Di Simone Andrea Luca

Report sperimentale Algorithm Engineering

Indice

1	Introduzione	3
2	Background teorico	4
2.1	Definizioni fondamentali	4
2.2	Classificazione degli algoritmi di massimo flusso	4
2.3	Algoritmi basati su cammini aumentanti	4
2.4	Algoritmi Preflow-Push (Push-Relabel)	4
2.5	Algoritmi con scaling delle capacità	5
2.6	Algoritmi quasi-lineari	5
3	Algoritmo Push-Relabel	6
3.1	Descrizione generale	6
3.2	Strutture dati	6
3.3	Operazioni fondamentali	6
3.4	Selezione dei nodi attivi: coda FIFO	7
3.5	Euristiche implementate	7
3.5.1	BFS inversa per l'inizializzazione delle altezze	7
3.5.2	Gap heuristic	8
3.5.3	Current-arc (advance pointer)	9
3.6	Complessità	10
4	Algoritmo Capacity Scaling	11
4.1	Descrizione generale	11
4.2	Strutture dati	11
4.3	Operazioni fondamentali	11
4.4	Struttura a fasi di scaling	12
4.5	Complessità	12
4.6	Gestione di capacità non intere	13
4.7	Confronto con Push-Relabel	14
5	Algoritmo Almost-Linear Time (IPM + Howard)	15
5.1	Descrizione generale	15
5.2	Riduzione da max-flow a min-cost flow	15
5.3	Funzione potenziale $\Phi(f)$	15
5.4	Il sottoproblema: minimum ratio cycle	16
5.5	Loop di ottimizzazione interior-point	16
5.6	Ricerca binaria sul valore ottimale	17
5.7	Algoritmo di Howard per il minimum cycle ratio	17
5.8	Strutture dati	18
5.9	Complessità	18
5.10	Gestione di capacità non intere e problemi numerici	19
5.11	Confronto con altri algoritmi	19
6	Dataset e istanze di test	20
6.1	Istanze random	20
6.2	Struttura del grafo Layered	20
6.3	Reti reali	21
7	Setup sperimentale	21
7.1	Hardware e software	21

8	Risultati sperimentali	22
8.1	Confronto sui tempi di esecuzione	22
8.2	Impatto del tipo di capacità	22
8.3	Tipologie di grafi usate nel dettaglio	22
8.3.1	Struttura del grafo Erdős–Rényi DAG	22
8.3.2	Struttura del grafo Grid	23
8.3.3	Struttura del grafo Layered	23
8.4	Push-Relabel	24
8.4.1	Grafi Erdős–Rényi	24
8.4.2	Grafi a griglia bidimensionale	25
8.4.3	Grafi a layer	27
8.5	Capacity Scaling	29
8.5.1	Grafi Erdős–Rényi	29
8.5.2	Grafi a griglia bidimensionale	35
8.5.3	Grafo Layered	41
8.6	Almost-Linear Time	48
8.6.1	Grafi Erdős–Rényi	48
8.6.2	Grafi a griglia bidimensionale	49
8.6.3	Grafo Layered	50
9	Discussione	52
9.1	Push-Relabel	52
9.2	Capacity scaling	53
9.3	algoritmi quasi-lineari	59
10	Conclusioni	60
11	Repository del codice	61
11.1	Riferimenti bibliografici	62

1 Introduzione

Il problema del massimo flusso in una rete costituisce uno dei temi fondamentali nell'ambito dell'ottimizzazione combinatoria e dell'Algorithm Engineering. Una rete di flusso permette di modellare in modo naturale una vasta gamma di problemi reali, che spaziano dal *routing* nelle reti di comunicazione allo *scheduling* di processi, dalla visione artificiale alla progettazione di infrastrutture di trasporto, fino all'analisi di reti energetiche e sociali. Come riportato nel documento originale, “il problema del massimo flusso in una rete trova applicazioni in ambiti quali routing, scheduling, visione artificiale, trasporto e progettazione di reti”, evidenziando la centralità di questo paradigma computazionale.

Nonostante la formulazione matematica sia relativamente compatta, il comportamento pratico degli algoritmi di massimo flusso dipende in modo critico da molteplici fattori: la struttura del grafo, la distribuzione delle capacità, la presenza di colli di bottiglia, la densità della rete e l'adozione di specifiche euristiche. Per tali ragioni, l'analisi sperimentale riveste un ruolo essenziale nel valutare l'effettiva efficienza degli algoritmi, andando oltre i soli risultati teorici.

L'obiettivo di questo report è analizzare in profondità il comportamento pratico di diversi algoritmi per il calcolo del massimo flusso, confrontandone prestazioni, robustezza e sensibilità alle caratteristiche delle istanze. Il documento presenta una panoramica teorica degli algoritmi considerati, descrive i dataset utilizzati, illustra la metodologia sperimentale e discute i risultati ottenuti, con particolare attenzione al tempo di esecuzione, al numero di operazioni elementari e all'impatto del tipo di capacità.

Lo studio sperimentale è stato progettato per fornire una valutazione completa e multidimensionale. Sono state considerate istanze random generate secondo diversi modelli (Erdős–Rényi, Barabási–Albert, griglie), reti reali provenienti da SNAP, DIMACS e power grid test cases, nonché istanze con caratteristiche peculiari quali capacità unitarie, capacità floating point e presenza di archi paralleli. Per ogni algoritmo sono state misurate metriche quali il tempo di esecuzione, il numero di operazioni elementari, il numero di fasi di scaling e la stabilità dei risultati su esecuzioni ripetute.

L'analisi combinata di teoria e sperimentazione consente di evidenziare differenze significative tra gli algoritmi, mostrando come alcune tecniche — in particolare le euristiche del metodo Push–Relabel — possano migliorare drasticamente le prestazioni pratiche, mentre altre, come gli algoritmi quasi-lineari, risultano più complessi da implementare e meno competitivi su istanze di dimensione moderata.

Il documento presenta una panoramica teorica degli algoritmi considerati, descrive i dataset utilizzati, illustra la metodologia sperimentale e discute i risultati ottenuti, con particolare attenzione al tempo di esecuzione, al numero di operazioni elementari e all'impatto del tipo di capacità.

2 Background teorico

2.1 Definizioni fondamentali

Una rete di flusso è un grafo orientato $G = (V, E)$ dotato di una funzione di capacità $c : E \rightarrow \mathbb{R}_{>0}$, una sorgente $s \in V$ e un pozzo $t \in V$. Un flusso è una funzione $f : E \rightarrow \mathbb{R}$ che soddisfa:

- **Vincolo di capacità:** $0 \leq f(u, v) \leq c(u, v)$ per ogni arco $(u, v) \in E$.
- **Conservazione del flusso:** per ogni $v \neq s, t$ vale

$$\sum_{u:(u,v) \in E} f(u, v) = \sum_{w:(v,w) \in E} f(v, w).$$

Il valore del flusso è definito come la quantità totale inviata dalla sorgente al pozzo:

$$|f| = \sum_{(s,v) \in E} f(s, v).$$

2.2 Classificazione degli algoritmi di massimo flusso

Gli algoritmi per il calcolo del massimo flusso possono essere suddivisi in diverse famiglie, ciascuna caratterizzata da un differente modello computazionale e da specifiche proprietà teoriche e pratiche.

2.3 Algoritmi basati su cammini aumentanti

Questa famiglia include metodi come Ford–Fulkerson ed Edmonds–Karp. L'idea centrale è individuare ripetutamente un cammino aumentante nella rete residua e inviare flusso lungo tale cammino. La complessità dipende fortemente dalla scelta del cammino.

2.4 Algoritmi Preflow–Push (Push–Relabel)

Gli algoritmi di tipo Preflow–Push, introdotti da Goldberg e Tarjan, mantengono un *preflow* e un'etichetta di altezza per ogni nodo. Le operazioni fondamentali sono:

- **push:** invio di flusso lungo archi ammissibili;
- **relabel:** aumento dell'altezza di un nodo per sbloccare il flusso.

Varianti importanti includono:

- Highest Label,
- FIFO,
- Gap heuristic,
- Global relabeling.

2.5 Algoritmi con scaling delle capacità

Gli algoritmi di tipo *capacity scaling* limitano inizialmente il flusso a capacità elevate e riducono progressivamente la soglia di scaling. Questo approccio riduce il numero di cammini aumentanti “piccoli” e migliora le prestazioni su grafi con capacità molto variabili.

2.6 Algoritmi quasi-lineari

Gli algoritmi quasi-lineari combinano concetti di metodi di punto interno, raggiungono complessità quasi-lineare in teoria, ma richiedono strutture dati avanzate e solutori numerici sofisticati, risultando complessi da implementare e raramente utilizzati in applicazioni pratiche.

3 Algoritmo Push-Relabel

3.1 Descrizione generale

L'algoritmo Push-Relabel, introdotto da Goldberg e Tarjan nel 1988 [1], risolve il problema del flusso massimo su una rete orientata $G = (V, E)$ con capacità $c : E \rightarrow \mathbb{R}_{\geq 0}$, sorgente s e pozzo t .

A differenza degli algoritmi basati su cammini aumentanti (Ford-Fulkerson, Edmonds-Karp), Push-Relabel opera *localmente*: mantiene un **pre-flusso**, ovvero una funzione f che può violare la conservazione del flusso nei nodi interni, accumulando un *eccesso* $e(u) = \sum_v f(v, u) - \sum_v f(u, v) \geq 0$. L'algoritmo termina quando tutti gli eccessi nei nodi interni sono stati azzerati.

3.2 Strutture dati

L'implementazione si basa sulle seguenti strutture:

- **Grafo residuo**: per ogni arco (u, v) con capacità $c(u, v)$, si mantengono un arco *forward* con capacità residua $c(u, v) - f(u, v)$ e un arco *reverse* con capacità $f(u, v)$. Ogni arco è rappresentato come una tripla $[to, rev_pos, res_cap]$, dove rev_pos è l'indice dell'arco inverso nella lista di adiacenza del nodo destinazione.
- **Funzione di altezza** $h : V \rightarrow \mathbb{N}$: assegna a ogni nodo un'etichetta intera. Un arco (u, v) è *ammissibile* se e solo se $h(u) = h(v) + 1$ e la sua capacità residua è positiva.
- **Eccesso** $e : V \rightarrow \mathbb{R}_{\geq 0}$: quantità di flusso in eccesso accumulata in ogni nodo.
- **Contatore** cnt : array dove $cnt[k]$ tiene il numero di nodi con altezza esattamente k . Utilizzato dall'euristica del gap.

3.3 Operazioni fondamentali

Push. Se un nodo u ha eccesso $e(u) > 0$ e esiste un arco ammissibile (u, v) con capacità residua $r(u, v) > 0$, si invia una quantità di flusso $\delta = \min(e(u), r(u, v))$ da u a v :

$$r(u, v) \leftarrow r(u, v) - \delta, \quad r(v, u) \leftarrow r(v, u) + \delta, \quad e(u) \leftarrow e(u) - \delta, \quad e(v) \leftarrow e(v) + \delta.$$

Relabel. Se un nodo u ha eccesso $e(u) > 0$ ma nessun arco ammissibile è disponibile (ovvero tutti gli archi residui uscenti portano a nodi con altezza $\geq h(u)$), si incrementa l'altezza di u :

$$h(u) \leftarrow 1 + \min_{(u, v) \in E_r} h(v),$$

dove E_r è l'insieme degli archi con capacità residua positiva. Questo garantisce che almeno un arco ammissibile diventi disponibile dopo l'operazione.

Inizializzazione. All'avvio si fissa $h(s) = |V|$ e $h(v) = 0$ per ogni $v \neq s$. Tutti gli archi uscenti dalla sorgente s vengono saturati (*pre-saturazione*): per ogni arco (s, v) con $c(s, v) > 0$ si invia un flusso pari all'intera capacità, creando eccesso nei vicini diretti di s .

3.4 Selezione dei nodi attivi: coda FIFO

L'implementazione adotta una **coda FIFO** per la gestione dei nodi attivi (nodi con eccesso positivo, esclusi s e t). Un nodo viene estratto dalla testa della coda, *scaricato* completamente tramite operazioni di push e relabel (*discharge loop*), e reinserito in coda solo se al termine del discharge mantiene ancora eccesso positivo. I nodi che ricevono flusso durante un push e non erano già in coda vengono accodati in fondo.

La selezione FIFO garantisce una complessità nel caso peggiore di $O(V^3)$ [1].

3.5 Euristiche implementate

3.5.1 BFS inversa per l'inizializzazione delle altezze

Motivazione. L'inizializzazione standard pone $h(v) = 0$ per ogni $v \neq s$. Questa è una stima molto approssimativa: i nodi lontani dal pozzo richiedono molti relabel prima che le loro altezze convergano ai valori utili. Ogni relabel costa $O(\deg(u))$ nel caso peggiore, e su grafi grandi o sparsi il numero di relabel necessari prima della convergenza può dominare il tempo di esecuzione.

Descrizione. Prima dell'inizio del ciclo principale, si esegue una **visita BFS sul grafo residuo inverso**, partendo dal pozzo t . Il grafo residuo inverso è costruito considerando solo gli archi con capacità residua positiva e invertendone la direzione: per ogni arco $(u, v) \in E_r$ con $r(u, v) > 0$ si aggiunge l'arco (v, u) al grafo inverso.

La BFS calcola la distanza esatta di ogni nodo da t nel grafo residuo, che costituisce un'altezza ammissibile ottimale al momento dell'inizializzazione:

$$h(v) = d_r(v, t), \quad h(s) = |V|.$$

I nodi irraggiungibili dal pozzo nel grafo residuo vengono inizializzati a $|V| + 1$, rendendoli immediatamente non operativi.

Effetto pratico. Con altezze iniziali già ottimali, il numero totale di operazioni di relabel si riduce drasticamente. Su grafi sparsi con molti nodi, il guadagno è spesso di un ordine di grandezza rispetto all'inizializzazione standard, come confermato dai risultati sperimentali (sezione ??).

Implementazione.

```
std::vector<int> PushRelabel::_bfs_heights_int(int t) const {
    const int n = _n;
    std::vector<int> h(n, 2 * n); // altezza sentinella per nodi non raggiunti
    h[t] = 0;

    // Costruzione del grafo inverso: radj[v] = nodi u tali che esiste (u->v) con cap > 0
    std::vector<std::vector<int>> radj(n);
    for (int u = 0; u < n; ++u)
        for (auto& e : _adj_int[u])
            if (e.cap > 0)
```



```

    radj[e.to].push_back(u);

// BFS standard a partire dal pozzo
std::deque<int> queue;
queue.push_back(t);
while (!queue.empty()) {
    int v = queue.front(); queue.pop_front();
    for (int u : radj[v]) {
        if (h[u] == 2 * n) { // nodo non ancora visitato
            h[u] = h[v] + 1;
            queue.push_back(u);
        }
    }
}
return h;
}

```

3.5.2 Gap heuristic

Motivazione. Durante l'esecuzione dell'algoritmo, può accadere che nessun nodo abbia altezza pari a un certo valore k con $0 < k < |V|$. In tal caso, i nodi con altezza $h(v) > k$ non possono mai raggiungere il pozzo t (che ha altezza 0) attraverso archi ammissibili, poiché il “gap” nel livello k interrompe qualsiasi cammino verso t . Continuare a processare questi nodi è inutile.

Descrizione. Ogni volta che si esegue un relabel su un nodo u , si decrementa $cnt[h_{old}]$ (dove h_{old} è l'altezza precedente di u). Se $cnt[h_{old}] = 0$ e $0 < h_{old} < |V|$, si è creato un gap al livello h_{old} : tutti i nodi $v \neq s$ con $h_{old} < h(v) < |V|$ vengono portati all'altezza $|V| + 1$, escludendoli definitivamente dal calcolo:

$$cnt[h_{old}] = 0 \wedge 0 < h_{old} < |V| \implies \forall v \neq s : h_{old} < h(v) < |V| \Rightarrow h(v) \leftarrow |V| + 1.$$

I current-arc pointer dei nodi coinvolti vengono azzerati.

Effetto pratico. Il gap heuristic elimina intere componenti di nodi irraggiungibili senza doverle esplorare esplicitamente, riducendo in modo significativo il numero totale di operazioni. Sebbene non migliori il bound teorico $O(V^3)$, l'impatto pratico è molto rilevante, specialmente su grafi con struttura a livelli.

Implementazione.

```

if (cnt[old_h] == 0 && old_h > 0 && old_h < n) {
    for (int v = 0; v < n; ++v) {
        if (v != s && h[v] > old_h && h[v] < n) {
            cnt[h[v]]--;
        }
    }
}

```

```

        h[v] = n + 1;
        cnt[h[v]]++;
        cur[v] = 0; // reimposta current-arc
    }
}
}

```

3.5.3 Current-arc (advance pointer)

Motivazione. Senza ottimizzazioni, la procedura di discharge di un nodo u scorre ogni volta l'intera lista di adiacenza di u alla ricerca di un arco ammissibile. Se la lista contiene $\deg(u)$ archi, ogni discharge costa $O(\deg(u))$ nel caso peggiore, anche se la maggior parte degli archi non è ammissibile.

Descrizione. Si mantiene per ogni nodo u un **puntatore all'arco corrente** $cur[u]$, inizializzato a 0. Durante il discharge di u :

- Se l'arco $cur[u]$ è ammissibile, si esegue il push e si rimane su quell'arco.
- Se l'arco non è ammissibile, si avanza il puntatore: $cur[u] \leftarrow cur[u] + 1$.
- Se il puntatore raggiunge la fine della lista ($cur[u] = \deg(u)$), tutti gli archi sono stati esaminati senza trovare push ammissibili: si esegue il relabel e si azzerava il puntatore, $cur[u] \leftarrow 0$.

Il puntatore non viene mai fatto retrocedere (salvo dopo un relabel), perché dopo un push su (u, v) l'arco rimane ammissibile finché $e(u) > 0$ oppure $r(u, v) = 0$; gli archi precedenti non possono essere diventati ammissibili nel frattempo.

Costo ammortizzato. Ogni arco viene percorso dal puntatore al più una volta per ogni relabel di u . Poiché il numero totale di relabel è $O(V^2)$, il costo totale dei movimenti del puntatore è $O(V^2 \cdot \deg_{\max}) = O(VE)$, ammortizzato su tutta l'esecuzione. Nella pratica, questo riduce drasticamente il numero di esaminazioni inutili degli archi.

Implementazione.

```

// senza current-arc: reset a ogni discharge
    cur[u] = 0;

// durante il discharge
if (cur[u] == static_cast<int>(adj[u].size())) {
    // relabel
    ...
    cur[u] = 0;          // reset dopo relabel
} else {
    if (res > 0 && h[u] == h[v] + 1) {
        // Arco ammissibile: invia min(excess[u], res) unità
    }
}

```

```

long long delta = std::min(excess[u], res);
edge.cap          -= delta;
adj[v][edge.rev].cap += delta;
excess[u]          -= delta;
excess[v]          += delta;
// Accoda v se diventa attivo
if (v != s && v != t && !in_queue[v] && excess[v] > 0) {
    queue.push_back(v); in_queue[v] = true;
}
} else {
    cur[u]++;
}

```

3.6 Complessità

Configurazione	Complessità teorica	Note
FIFO puro	$O(V^3)$	Bound teorico, Goldberg & Tarjan 1988
+ Gap heuristic	$O(V^3)$	Stesso bound, ma drasticamente migliore in pratica
+ Current-arc	$O(V^2\sqrt{E})$	Con selezione highest-label; FIFO mantiene $O(V^3)$
+ BFS init	$O(V^3)$	Riduce il numero assoluto di relabel

Table 1: Complessità dell'algoritmo Push-Relabel nelle diverse configurazioni.

4 Algoritmo Capacity Scaling

4.1 Descrizione generale

L'algoritmo Capacity Scaling, introdotto da Gabow nel 1985 [2], risolve il problema del flusso massimo su una rete orientata $G = (V, E)$ con capacità $c : E \rightarrow \mathbb{R}_{>0}$, sorgente s e pozzo t .

A differenza dell'algoritmo di Edmonds-Karp, che aumenta il flusso lungo qualsiasi cammino aumentante nella rete residua, Capacity Scaling introduce il concetto di **fase di scaling**: in ogni fase si considera solo la porzione della rete residua in cui gli archi hanno capacità residua almeno pari a una soglia Δ , chiamata Δ -grafo residuo. La soglia viene dimezzata progressivamente fino a zero, garantendo che in ogni fase il numero di aumentazioni sia limitato.

4.2 Strutture dati

L'implementazione si basa sulle seguenti strutture:

- **Grafo residuo**: per ogni arco (u, v) con capacità $c(u, v)$, si mantengono un arco *forward* con capacità residua $r(u, v) = c(u, v) - f(u, v)$ e un arco *reverse* con capacità $r(v, u) = f(u, v)$. Ogni arco è rappresentato come una tripla $[to, rev_pos, res_cap]$.
- **Soglia Δ** : valore che determina quali archi sono ammissibili in una data fase. Solo gli archi con $r(u, v) \geq \Delta$ vengono considerati.
- **Tabella parent**: struttura restituita dalla BFS che codifica il cammino aumentante trovato, nella forma $\{nodo \rightarrow (nodo_padre, indice_arco)\}$.

4.3 Operazioni fondamentali

BFS nel Δ -grafo residuo. Data la soglia corrente Δ , si esegue una visita BFS sulla rete residua considerando solo gli archi con capacità residua $r(u, v) \geq \Delta$. Se il pozzo t è raggiungibile dalla sorgente s , la BFS restituisce la tabella *parent* che codifica un cammino aumentante; altrimenti restituisce **None** e la fase corrente termina. Il costo è $O(m)$.

Augmentazione. Dato un cammino aumentante codificato in *parent*, si procede in due passi:

1. Si percorre il cammino a ritroso da t a s per trovare il **bottleneck**:

$$\delta = \min_{(u,v) \in P} r(u, v).$$

2. Si percorre nuovamente il cammino a ritroso aggiornando le capacità residue:

$$r(u, v) \leftarrow r(u, v) - \delta, \quad r(v, u) \leftarrow r(v, u) + \delta.$$

Il flusso inviato δ viene sommato al totale. Il costo è $O(n)$.

4.4 Struttura a fasi di scaling

Inizializzazione di Δ . Si sceglie come valore iniziale di Δ la più grande potenza di 2 non superiore alla massima capacità $U = \max_{(u,v) \in E} c(u,v)$:

$$\Delta_0 = 2^{\lfloor \log_2 U \rfloor}.$$

Nell'implementazione, poiché le capacità possono essere valori reali (irrazionali o razionali), si usa `math.floor` e `math.log2` invece dell'operatore di shift intero:

```
delta = 2 ** math.floor(math.log2(U))
```

Ciclo principale. L'algoritmo procede secondo il seguente schema:

```
while (delta >= EPS) {
    // Fase: aumenta finché esistono cammini nel delta-grafo residuo
    while (true) {
        auto path = _bfs_dbl(s, t, delta);
        if (!path.has_value()) break;
        total += _augment_dbl(s, t, *path);
    }
    delta /= 2.0; // prossima fase con soglia dimezzata
}
return total;
```

Il ciclo esterno scandisce le fasi di scaling: $\Delta_0, \Delta_0/2, \Delta_0/4, \dots$. Il ciclo interno esegue tutte le aumentazioni possibili nel Δ -grafo residuo corrente. Quando non esistono più cammini aumentanti con capacità residua almeno Δ , si dimezza Δ e si passa alla fase successiva.

Condizione di terminazione. Per capacità intere, l'algoritmo termina quando $\Delta < 1$. Per capacità reali (razionali o irrazionali), si usa una soglia numerica $\varepsilon > 0$ per evitare iterazioni infinite dovute all'aritmetica floating point:

$$\text{EPS} = 10^{-9}.$$

Il ciclo prosegue finché $\Delta \geq \varepsilon$.

4.5 Complessità

Numero di fasi. Il numero totale di fasi è $\lfloor \log_2 U \rfloor + 1 = O(\log U)$, dove U è la massima capacità degli archi.

Augmentazioni per fase. In ogni fase con soglia Δ , ogni augmentazione invia almeno Δ unità di flusso. Il flusso massimo è al più $|V| \cdot U$ (somma delle capacità degli archi uscenti da s), quindi il numero di augmentazioni per fase è al più $O(m)$, poiché ogni augmentazione satura almeno un arco nel Δ -grafo residuo e ogni arco può essere saturato al più due volte (nella direzione forward e reverse) prima di uscire dal Δ -grafo.

Costo totale. Ogni augmentazione richiede una BFS di costo $O(m)$ e un'augmentazione di costo $O(n)$. Dominando la BFS, il costo per fase è $O(m \cdot m) = O(m^2)$. Su $O(\log U)$ fasi:

$$T(n, m, U) = O(m^2 \log U).$$

Parametro	Valore
Numero di fasi	$O(\log U)$
Augmentazioni per fase	$O(m)$
Costo per augmentazione (BFS)	$O(m)$
Complessità totale	$O(m^2 \log U)$

Table 2: Complessità dell'algoritmo Capacity Scaling.

4.6 Gestione di capacità non intere

L'implementazione estende l'algoritmo originale, concepito per capacità intere, al caso di capacità razionali e irrazionali.

Capacità razionali con scaling. Quando il programma viene lanciato le capacità vengono convertite in interi esatti prima dell'esecuzione tramite la funzione `scale_rationals`: si calcola il minimo comune multiplo k dei denominatori di tutte le capacità e si moltiplicano tutte le capacità per k . Il flusso risultante viene diviso per k al termine:

$$f_{\text{reale}} = \frac{f_{\text{intero}}}{k}.$$

In questo modo l'algoritmo opera interamente su interi, preservando la correttezza esatta.

Capacità irrazionali Le capacità vengono trattate direttamente come valori `float`. I due adattamenti principali rispetto alla versione intera sono:

- **Calcolo di Δ_0 :** si sostituisce l'operatore di shift intero `U.bit_length()` con `math.floor(math.log2(U))`, che funziona correttamente su valori `float`.
- **Dimezzamento di Δ :** si sostituisce lo shift `delta >= 1` con la divisione `delta /= 2`, compatibile con `float`.
- **Condizione di terminazione:** la condizione `delta >= 1` viene sostituita da `delta >= EPS` con `EPS = 10-9`, per evitare loop infiniti causati da capacità residue numericamente non nulle ma trascurabili.
- **Soglia nella BFS:** gli archi vengono inclusi nel Δ -grafo residuo se $r(u, v) \geq \Delta$ (confronto diretto tra `float`).

L'uso di capacità `float` introduce inevitabilmente errori di arrotondamento. Per capacità irrazionali, il valore del flusso massimo calcolato è un'approssimazione floating point del valore esatto; la precisione è determinata dall'accumulo degli errori durante le augmentazioni.

4.7 Confronto con Push-Relabel

Proprietà	Capacity Scaling	Push-Relabel (FIFO)
Complessità teorica	$O(m^2 \log U)$	$O(V^3)$
Dipendenza da U	Sì	No
Strategia	Cammini aumentanti	Pre-flusso locale
Capacità reali	Supportate (via EPS)	Supportate (via EPS)
Capacità razionali esatte	Via scaling a interi	Via scaling a interi

Table 3: Confronto tra Capacity Scaling e Push-Relabel.

Dal punto di vista teorico Capacity Scaling è preferibile quando $m \ll V^2$ (grafi sparsi) e $\log U$ è piccolo. Push-Relabel è preferibile su grafi densi o quando le capacità sono molto grandi.

5 Algoritmo Almost-Linear Time (IPM + Howard)

5.1 Descrizione generale

L'algoritmo Almost-Linear Time, introdotto da Chen, Kyng, Liu, Peng, Gutenberg e Sachdeva nel 2022 [3], risolve il problema del massimo flusso e del minimo costo in tempo $m^{1+o(1)}$, dove $m = |E|$ è il numero di archi. La complessità quasi-lineare lo rende asintoticamente superiore agli algoritmi classici (Edmonds-Karp, Push-Relabel, Capacity Scaling) su istanze grandi.

L'algoritmo si basa su due componenti principali:

1. Un **metodo interior-point a riduzione di potenziale** (IPM) che riduce il problema di min-cost flow a una sequenza di $m^{1+o(1)}$ sottoproblemi di *minimum-ratio cycle*.
2. Una **struttura dati dinamica e randomizzata** per risolvere ciascun sottoproblema in tempo ammortizzato $m^{o(1)}$.

Nell'implementazione descritta in questo capitolo si adotta un approccio *statico*: la struttura dati dinamica è sostituita dall'**algoritmo di Howard** [4], un oracolo esatto per il minimum cycle ratio (MCR), lasciando invariata la logica dell'IPM. Questo permette di valutare la correttezza e la praticabilità dell'IPM in isolamento, indipendentemente dalla complessità implementativa della struttura dati originale.

5.2 Riduzione da max-flow a min-cost flow

La riduzione standard da max-flow a min-cost flow consiste nell'assegnare costo 0 a tutti gli archi originali e aggiungere un arco virtuale ($t \rightarrow s$) con costo -1 e capacità sufficientemente grande. Un algoritmo di min-cost flow minimizzerà il costo complessivo massimizzando il flusso sull'arco virtuale, che coincide con il valore del massimo flusso s - t .

Formalmente, data un'istanza di max-flow su $G = (V, E)$ con capacità $c : E \rightarrow \mathbb{Z}_{>0}$, si costruisce l'istanza di min-cost flow $\hat{G}$ come segue:

- Per ogni arco $e \in E$: costo $c_e = 0$, capacità inferiore $u_e^- = 0$, capacità superiore $u_e^+ = c_e$.
- Arco virtuale ($t \rightarrow s$): costo -1 , $u_{ts}^- = 0$, $u_{ts}^+ = \sum_{v:(s,v) \in E} c_{sv}$.
- Costo ottimale noto: $F^* = -f^*$, dove f^* è la stima corrente del flusso massimo.

5.3 Funzione potenziale $\Phi(f)$

Il cuore dell'IPM è la funzione potenziale a riduzione di potenziale:

$$\Phi(f) = 20m \log(c^\top f - F^*) + \sum_{e \in E} \left[(u_e^+ - f_e)^{-\alpha} + (f_e - u_e^-)^{-\alpha} \right]$$

dove $\alpha = 1/\log_2(1000 \cdot m \cdot U)$, $U = \max_e |u_e^+|$, e F^* è il costo ottimale (assunto noto dalla ricerca binaria, cfr. Sezione 5.6).

Il potenziale combina:

- **Obiettivo**: il termine $20m \log(c^\top f - F^*)$ decresce al diminuire del costo del flusso.

- **Barriera:** la somma $\sum_e (\cdot)^{-\alpha}$ penalizza le soluzioni prossime ai bordi del dominio ammissibile $u^- < f < u^+$, impedendo la violazione dei vincoli di capacità. La barriera di potenza $x^{-\alpha}$ con $0 < \alpha \ll 1$ è meno restrittiva della barriera logaritmica standard $-\log x$: lascia avvicinare il flusso ai limiti di capacità fino a valori estremamente prossimi a zero prima di divergere, garantendo maggiore precisione sulla soluzione finale.

Gradienti e lunghezze. Il gradiente di Φ rispetto a f_e è:

$$g(f)_e = \nabla_e \Phi(f) = 20m (c^\top f - F^*)^{-1} c_e + \alpha (u_e^+ - f_e)^{-1-\alpha} - \alpha (f_e - u_e^-)^{-1-\alpha}$$

La lunghezza di un arco misura quanto il flusso è vincolato dalle capacità:

$$\ell(f)_e = (u_e^+ - f_e)^{-1-\alpha} + (f_e - u_e^-)^{-1-\alpha}$$

5.4 Il sottoproblema: minimum ratio cycle

Ad ogni iterazione dell'IPM occorre trovare una circolazione $\Delta \in \mathbb{R}^E$ (con $B^\top \Delta = 0$) che minimizzi il rapporto:

$$\min_{B^\top \Delta = 0} \frac{g^\top \Delta}{\|L\Delta\|_1}$$

dove $L = \text{diag}(\ell)$. Il ciclo di minimo rapporto fornisce la direzione di discesa ottimale per $\Phi(f)$: il numeratore negativo indica che muoversi lungo Δ riduce il potenziale, mentre il denominatore cresce all'avvicinarsi ai bordi del dominio, limitando l'ampiezza del passo.

Nell'implementazione, il sottoproblema è risolto dall'**algoritmo di Howard** (cfr. Sezione 5.7), che opera sul grafo non orientato trattando ogni arco $e = (u, v)$ come percorribile in entrambe le direzioni: se percorso in direzione opposta, il gradiente dell'arco cambia segno ($-g_e$), così come la componente di Δ .

5.5 Loop di ottimizzazione interior-point

Flusso ammissibile iniziale. L'IPM richiede un punto di partenza strettamente interno al dominio ammissibile di Φ . Si costruisce il flusso centrale $f_0 = (u^- + u^+)/2$, che soddisfa per costruzione i vincoli di capacità ma in generale viola la conservazione del flusso ($B^\top f_0 \neq 0$).

Per correggere gli squilibri nodali $\hat{d} = B^\top f_0$, si aggiunge un nodo fittizio v^* collegato a ciascun nodo sbilanciato:

- Se $\hat{d}_v > 0$ (eccesso su v): arco $v^* \rightarrow v$ con flusso iniziale $\hat{d}_v$ e capacità superiore $2\hat{d}_v$.
- Se $\hat{d}_v < 0$ (deficit su v): arco $v \rightarrow v^*$ con flusso iniziale $|\hat{d}_v|$ e capacità superiore $2|\hat{d}_v|$.

Il costo degli archi ausiliari è $c = 4mU^2$, sufficientemente elevato da renderli non attraenti per l'ottimizzatore, così da non influire sulla soluzione dell'istanza originale. Il flusso iniziale f_0 esteso alla nuova istanza soddisfa la conservazione del flusso e appartiene all'interno del dominio ammissibile di Φ .

Ciclo principale. Il loop di ottimizzazione è strutturato come segue:

```
while c·f - F* >= threshold:
    g = calc_gradients(f)
    l = calc_lengths(f)
    (min_ratio, Delta) = howard(I, g, l)
    assert min_ratio < 0
    eta = -(kappa^2) / (50 * g · Delta)
    f = f + eta * upscale * Delta
    assert Phi(f_new) < Phi(f_old)
```

Lo **step-size** η è calcolato come:

$$\eta = -\frac{\kappa^2}{50 \cdot g^\top \Delta}$$

con $\kappa \in (0, 1)$ fattore di qualità dell'approssimazione. Poiché $g^\top \Delta < 0$, risulta $\eta > 0$. Il fattore **upscale** (tipicamente 10^3) amplifica il passo permettendo una convergenza più rapida, a scapito di una maggiore esposizione al rischio di violazione dei vincoli su istanze grandi.

Le invarianti verificate ad ogni iterazione sono:

1. Conservazione del flusso: $\|B^\top f\|_\infty < 10^{-10}$.
2. Ratio negativo: $\text{min_ratio} < 0$.
3. Decrescita stretta del potenziale: $\Phi(f_{\text{new}}) < \Phi(f_{\text{old}})$.

Il loop termina quando $c^\top f - F^* < \varepsilon$ (con $\varepsilon = 10^{-5}$) oppure quando $\Phi(f)$ non è più finito (bordo del dominio raggiunto per errori numerici).

5.6 Ricerca binaria sul valore ottimale

La funzione potenziale $\Phi(f)$ richiede la conoscenza del costo ottimale F^* . Questo valore non è noto a priori: si stima tramite ricerca binaria nell'intervallo $[0, U_s]$, dove $U_s = \sum_{v:(s,v) \in E} c_{sv}$ è la somma delle capacità uscenti dalla sorgente.

Ad ogni iterazione binaria si testa un candidato mid:

- Se il flusso restituito da `max_flow_with_guess(mid)` è $< \text{mid}$: il valore non è raggiungibile, si abbassa **high**.
- Altrimenti: il valore è raggiungibile, si alza **low**.

La ricerca termina in $O(\log U_s)$ iterazioni, ciascuna delle quali invoca il loop IPM descritto nella Sezione 5.5.

5.7 Algoritmo di Howard per il minimum cycle ratio

L'algoritmo di Howard [4] è un metodo iterativo per il problema del minimum cycle ratio (MCR) su grafi pesati diretti (o non diretti). Dato un grafo $G = (V, E)$ con pesi g_e (gradiente) e $\ell_e > 0$ (lunghezza) per ogni arco, il MCR è definito come:

$$\lambda^* = \min_{\text{ciclo } C} \frac{\sum_{e \in C} g_e}{\sum_{e \in C} \ell_e}$$

Schema dell'algoritmo. Howard costruisce e raffina iterativamente una *policy*: un sottografo π in cui ogni nodo ha esattamente un arco uscente. Su tale policy si esegue il rilevamento di cicli; per ogni ciclo trovato si calcola il ratio e si aggiorna la policy scegliendo, per ciascun nodo, l'arco che minimizza localmente il ratio. Il processo converge al ciclo di ratio minimo.

Per supportare il grafo non orientato dell'IPM, l'implementazione considera ogni arco $e = (u, v)$ anche in direzione inversa (v, u) : il gradiente dell'arco percorso in senso inverso è $-g_e$, mentre la lunghezza rimane ℓ_e (sempre positiva). Il risultato di Howard è un vettore $\Delta \in \{-1, 0, +1\}^E$, dove $\Delta_e = +1$ se l'arco è percorso in avanti, $\Delta_e = -1$ se percorso all'indietro e $\Delta_e = 0$ se non appartiene al ciclo.

5.8 Strutture dati

- **MinCostFlow**: struttura centrale che incapsula il grafo, i vettori Eigen c , u^- , u^+ , la matrice di incidenza $B \in \mathbb{R}^{m \times n}$, i parametri U e α , e i metodi `phi`, `calc_gradients`, `calc_lengths`.
- **Matrice di incidenza B** : $B_{e,v} = +1$ se l'arco e esce da v , $B_{e,v} = -1$ se entra in v , 0 altrimenti. Permette di esprimere la conservazione del flusso come $B^\top f = 0$ e di calcolare la divergenza nodale in $O(m)$ con una moltiplicazione matrice-vettore.
- **Vettori Eigen (VectorXd)**: tutti i flussi, gradienti e lunghezze sono rappresentati come vettori `float64` di Eigen, che supporta nativamente prodotti scalari, norme e operazioni entrywise utilizzate nel calcolo di Φ , g e ℓ .
- **Lista di adiacenza `adj`**: per ogni nodo v , `adj[v]` contiene gli indici degli archi incidenti (entranti e uscenti), usata da Howard per la visita dei vicini.
- **Mappa `undirected_edge_to_indices`**: hash map da coppia di nodi (a, b) agli indici degli archi tra a e b , con lookup $O(1)$ ammortizzato (hash basato su codifica a 64 bit).

5.9 Complessità

Numero di iterazioni IPM. [CKLPGS22] dimostra che, con un oracolo MCR che restituisce un ciclo $m^{o(1)}$ -approssimato e $\kappa \geq m^{-o(1)}$, il potenziale $\Phi(f)$ decresce di $\Omega(\kappa^2)$ per iterazione. Partendo da $\Phi(f^{(0)}) \leq 200m \log(mU)$ e terminando a $\Phi(f) \leq -200m \log(mU)$, il numero totale di iterazioni è $O(m\kappa^{-2}) = m^{1+o(1)}$.

Costo per iterazione. Con la struttura dati dinamica di [CKLPGS22], ogni iterazione costa $m^{o(1)}$ ammortizzato, portando a un costo totale $m^{1+o(1)}$. Nell'implementazione statica qui descritta, Howard ha complessità per iterazione $O(nm)$ nel caso peggiore, rendendo la complessità pratica dell'implementazione significativamente più alta.

Ricerca binaria. La ricerca binaria aggiunge un fattore $O(\log U)$, portando la complessità totale teorica a $m^{1+o(1)} \log U$ per il max-flow.

Componente	Complessità
Iterazioni IPM	$m^{1+o(1)}$
Oracolo MCR (Howard, statico)	$O(nm)$ per iterazione
Ricerca binaria	$O(\log U)$ invocazioni IPM
Complessità totale (teorica, con DS)	$m^{1+o(1)} \log U$
Complessità totale (implementazione statica)	$O(nm^2 \log U)$

Table 4: Complessità dell'algoritmo Almost-Linear Time con oracolo statico (Howard).

5.10 Gestione di capacità non intere e problemi numerici

L'implementazione usa `long long` per le capacità degli archi (evitando overflow su istanze con capacità elevate) e `float64` (Eigen `VectorXd`) per i flussi e le quantità dell'IPM. Questa scelta diverge dal modello di calcolo a virgola fissa di [3], che garantisce la rappresentazione di tutti i valori intermedi in $O(\log^{O(1)} z)$ bit (con z dimensione dell'input in bit).

5.11 Confronto con altri algoritmi

Algoritmo	Complessità	Dipende da U	Praticità
Edmonds-Karp	$O(m^2 n)$	No	Alta
Capacity Scaling	$O(m^2 \log U)$	Sì	Alta
Push-Relabel (FIFO)	$O(n^3)$	No	Alta
Almost-Linear (IPM)	$m^{1+o(1)} \log U$	Sì	Bassa (costanti grandi)

Table 5: Confronto tra l'algoritmo Almost-Linear Time e algoritmi classici per il max-flow.

Come osservato sperimentalmente in [5], nonostante la complessità asintotica ottimale, l'IPM richiede circa 10^4 – 10^5 iterazioni su istanze con $m \leq 500$, contro le poche centinaia di aggiornamenti di Push-Relabel sulle stesse istanze. Le costanti nascoste nell'esponente $o(1)$ rendono l'algoritmo poco competitivo su istanze di dimensione pratica, confermando che il salto teorico non si traduce in un vantaggio pratico immediato.

6 Dataset e istanze di test

6.1 Istanze random

Sono stati generati grafi casuali utilizzando diversi modelli:

- Grafi Erdős–Rényi $G(n, p)$
- Grafi a layer

6.2 Struttura del grafo Layered

Il grafo layered è organizzato come una griglia di livelli (layer), generata secondo i seguenti passi:

1. **Dimensionamento della griglia.** Dati n nodi totali, si calcolano larghezza W e numero di livelli L in modo che il rapporto $L/W \approx 2$:

$$W = \left\lfloor \sqrt{n/2} \right\rfloor, \quad L = 2W$$

Il numero di livelli L non è quindi un parametro indipendente, ma è derivato automaticamente da n .

2. **Assegnazione dei nodi ai livelli.** I nodi sono numerati da 1 a n e distribuiti in ordine nei livelli: i primi W nodi nel livello 0, i successivi W nel livello 1, e così via. La sorgente s coincide con il nodo 1 (primo nodo del livello 0); il pozzo t coincide con il nodo n (ultimo nodo dell'ultimo livello).
3. **Generazione degli archi.** Per ogni nodo u di un livello ℓ :
 - vengono generati archi verso $\lfloor d/2 \rfloor$ nodi scelti casualmente *nello stesso livello* ℓ ;
 - vengono generati archi verso d nodi scelti casualmente nel *livello successivo* $\ell + 1$ (se esiste).

Ogni nodo ha quindi un grado medio in uscita pari a circa $1.5d$.

4. **Archivi terminali.** Tutti i nodi dell'ultimo livello $L - 1$ vengono collegati direttamente al pozzo t .
5. **Output.** Il grafo risultante viene serializzato in formato standard DIMACS (`.max`), con intestazione `p max <nodi> <archi>` e righe di arco `a u v capacità`.

- Grafi a griglia bidimensionale (con source nel nodo in alto a sinistra)

Sono state assegnate:

- capacità intere
- capacità frazionarie razionali
- capacità irrazionali (soglia a $1e-9$)
- capacità unitarie

Queste istanze permettono di valutare la sensibilità degli algoritmi alla natura delle capacità e alla struttura del grafo.

6.3 Reti reali

Sono state utilizzate reti di Stereo instances provenienti da:

[<https://vision.cs.uwaterloo.ca/data/maxflow>]

- BVZ - 4-connected grid w/ dense terminal arcs from source or to sink,
- KZ2 - two 4-connected grids w/ sparse connections between them (each vertex is connected to at most two vertices in the other grid); dense terminal arcs from source or to sink,

7 Setup sperimentale

7.1 Hardware e software

Gli esperimenti sono stati eseguiti su una macchina con le seguenti caratteristiche:

- **CPU:** *Ryzen 5 7600X 5.3 GHz*
- **RAM:** *32 GB DDR5 6000 MHz*
- **Sistema operativo:** *Windows 11*
- **Compilatore:** *MinGW*
- **Librerie utilizzate:** *Eigen*

8 Risultati sperimentali

8.1 Confronto sui tempi di esecuzione

In questa sezione vengono riportati i tempi di esecuzione dei diversi algoritmi su tutte le istanze considerate. I risultati mostrano differenze significative a seconda della struttura del grafo, della densità e del tipo di capacità. Sono stati prodotti grafici e tabelle per evidenziare:

- l'andamento del tempo di esecuzione al variare della dimensione del grafo;
- la sensibilità alle capacità intere, frazionarie, irrazionali e unitarie.

Queste misure permettono di correlare il comportamento empirico con la complessità teorica e di individuare eventuali colli di bottiglia.

Si specifica che per ogni singola combinazione di vertici, archi o capacità misurate sono state prese le mediane dei tempi di esecuzione di 3 seed, ciascuno con 3 esecuzioni a caldo prendendone la mediana, dopo 1 esecuzione a freddo non misurata.

8.2 Impatto del tipo di capacità

Sono stati analizzati quattro scenari principali:

- capacità intere nel range $[1, 1023]$;
- capacità frazionali numeratore nel range $[1, 1023]$ e denominatore nel range $[1, 127]$;
- capacità reali date da funzioni e convertite in floating point:
 - **Radici:** $\sqrt{n}$, $a\sqrt{n}$
 - **Logaritmi:** $\log(n)$, $a \log(n)$
 - **Esponenziali:** e^n
 - **Trigonometriche:** $\cos(\frac{\pi}{n})$, $\sin(\frac{\pi}{n})$, $\tan(\frac{\pi}{n})$
 - **Costanti:** $a\pi$, ae
- capacità unitarie

8.3 Tipologie di grafi usate nel dettaglio

8.3.1 Struttura del grafo Erdős–Rényi DAG

Il grafo ER-DAG è generato secondo il modello di Erdős–Rényi, adattato per garantire l'aciclicità:

1. **Costruzione base.** Per ogni coppia di nodi (u, v) con $u < v$, si inserisce l'arco $u \rightarrow v$ in modo indipendente con probabilità p . Poiché gli archi vanno sempre da indici minori a indici maggiori, il grafo risultante è aciclico per costruzione (DAG).
2. **Garanzie di connettività minima.** Dopo la generazione casuale, vengono applicati due controlli di sicurezza:
 - se il nodo sorgente 1 non ha alcun arco uscente, viene forzato l'arco $1 \rightarrow 2$;

- se il nodo pozzo n non ha alcun arco entrante, viene forzato l'arco $(n-1) \rightarrow n$.

Questo garantisce che esista sempre almeno un potenziale percorso parziale da s , anche per grafi molto sparsi.

8.3.2 Struttura del grafo Grid

Il grafo grid è una griglia regolare deterministica, generata come segue:

1. **Dimensionamento della griglia.** Dati n nodi totali e un parametro d , la griglia ha:

$$\text{righe} = d, \quad \text{colonne} = n/d$$

con il vincolo che n sia esattamente divisibile per d (altrimenti il generatore solleva un errore).

2. **Numerazione dei nodi.** I nodi sono numerati riga per riga (*row-major*): il nodo alla riga r , colonna c ha indice $r \cdot \text{cols} + c + 1$. La sorgente s è il nodo 1 (angolo in alto a sinistra); il pozzo t è il nodo n (angolo in basso a destra).
3. **Generazione degli archi.** A differenza dei layered e degli ER-DAG, qui la topologia è completamente deterministica: ogni nodo $u = (r, c)$ genera (se esistono):
 - un arco verso destra, al nodo $(r, c+1)$;
 - un arco verso il basso, al nodo $(r+1, c)$.

Non ci sono archi diagonali, intra-livello extra o connessioni casuali sulla topologia: la struttura è fissa per ogni (n, d) .

8.3.3 Struttura del grafo Layered

Il grafo layered è organizzato come una griglia di livelli (layer), generata secondo i seguenti passi:

1. **Dimensionamento della griglia.** Dati n nodi totali, si calcolano larghezza W e numero di livelli L in modo che il rapporto $L/W \approx 2$:

$$W = \left\lfloor \sqrt{n/2} \right\rfloor, \quad L = 2W$$

Il numero di livelli L non è quindi un parametro indipendente, ma è derivato automaticamente da n .

2. **Assegnazione dei nodi ai livelli.** I nodi sono numerati da 1 a n e distribuiti in ordine nei livelli: i primi W nodi nel livello 0, i successivi W nel livello 1, e così via. La sorgente s coincide con il nodo 1 (primo nodo del livello 0); il pozzo t coincide con il nodo n (ultimo nodo dell'ultimo livello).
3. **Generazione degli archi.** Per ogni nodo u di un livello ℓ :
 - vengono generati archi verso $\lfloor d/2 \rfloor$ nodi scelti casualmente *nello stesso livello* ℓ ;

- vengono generati archi verso d nodi scelti casualmente nel *livello successivo* $\ell + 1$ (se esiste).

Ogni nodo ha quindi un grado medio in uscita pari a circa $1.5d$.

4. **Archivi terminali.** Tutti i nodi dell'ultimo livello $L - 1$ vengono collegati direttamente al pozzo t .

8.4 Push-Relabel

8.4.1 Grafi Erdős–Rényi

n Numero totale di nodi.

p Probabilità di inserimento di ciascun arco $u \rightarrow v$ (con $u < v$). È il parametro effettivamente usato dalla funzione di generazione.

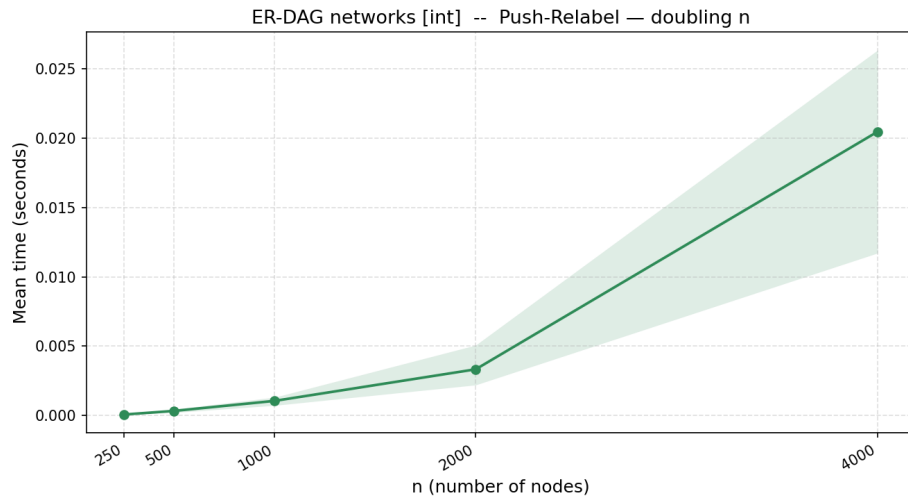


Figure 1: Push-Relabel — Erdag, capacità intere, $p = 0.02$

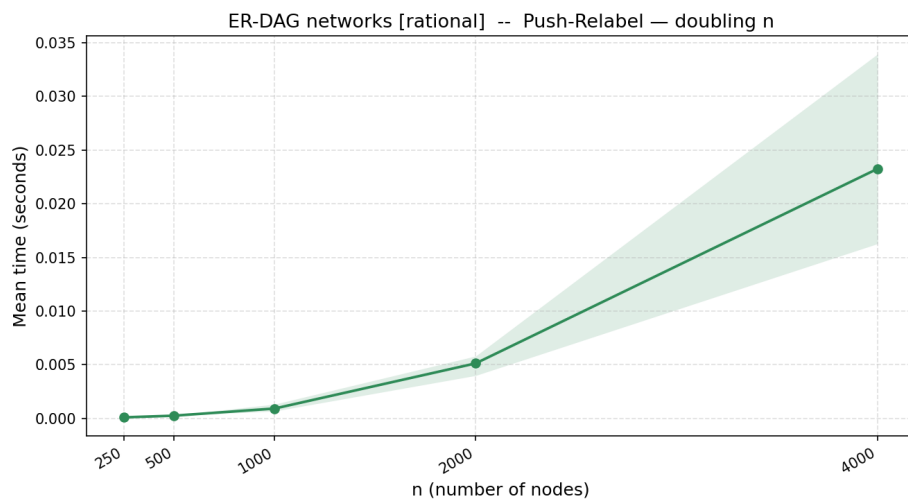


Figure 2: Push-Relabel — Erdag, capacità razionali, $p = 0.02$

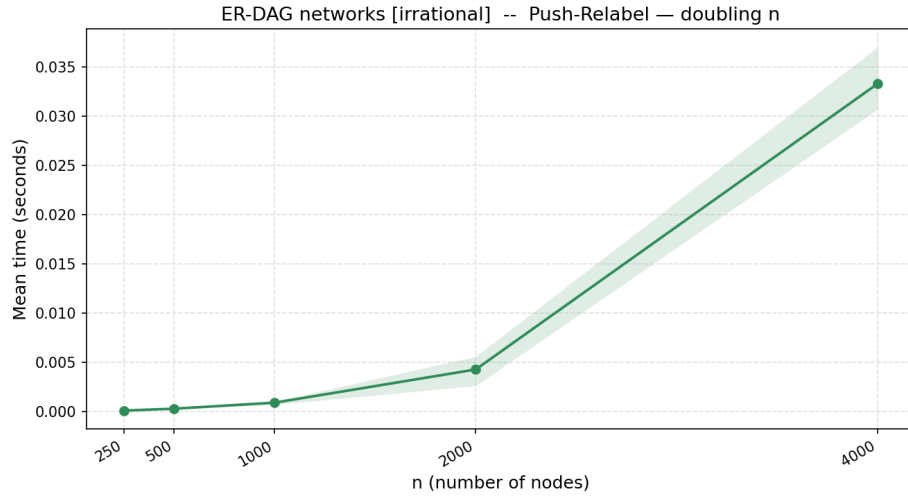


Figure 3: Push-Relabel — Erdag, capacità irrazionali, $p = 0.02$

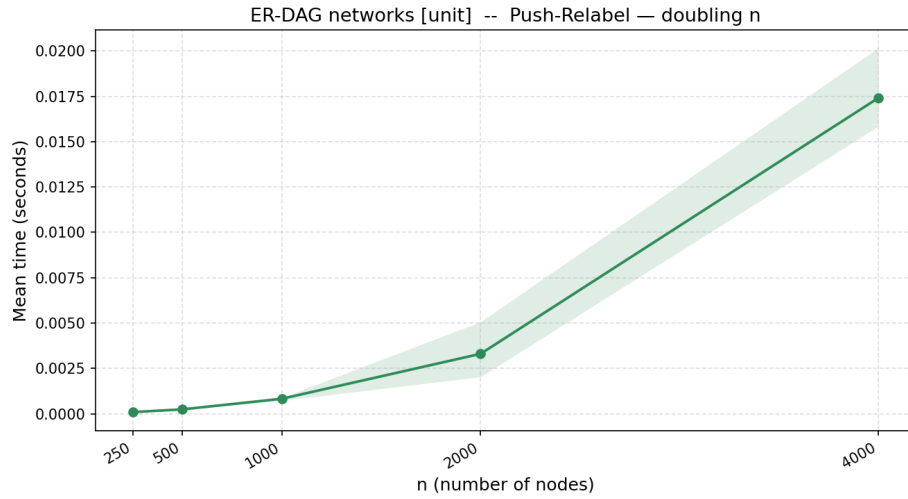


Figure 4: Push-Relabel — Erdag, capacità unitarie, $p = 0.02$

8.4.2 Grafi a griglia bidimensionale

n Numero totale di nodi. Deve essere divisibile per d .

d Numero di righe della griglia. Il numero di colonne è derivato come n/d . A differenza dei layered, d qui ha un significato strutturale diretto e univoco (non è un grado medio).

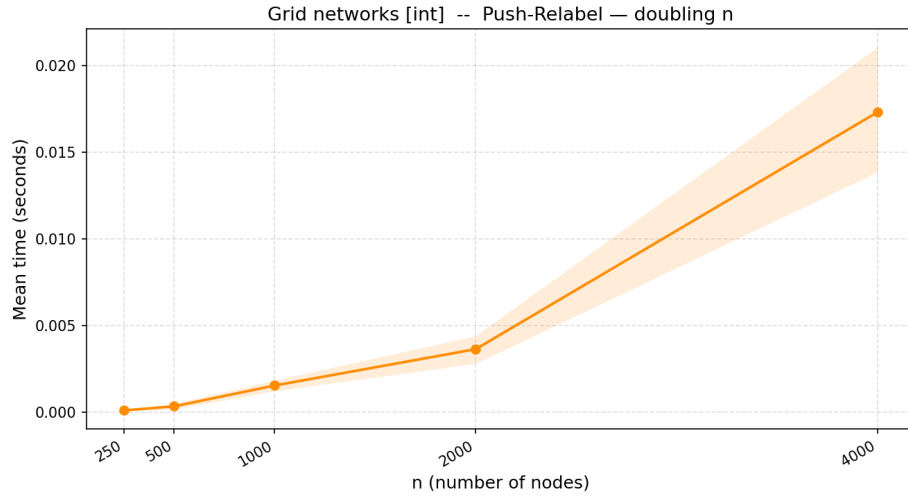


Figure 5: Push-Relabel — Grid, capacità intere, $d = 5$

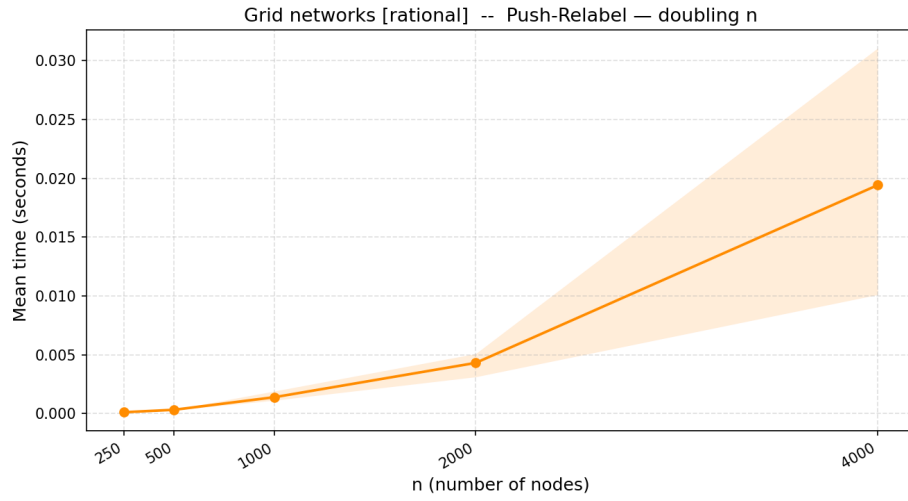


Figure 6: Push-Relabel — Grid, capacità razionali, $d = 5$

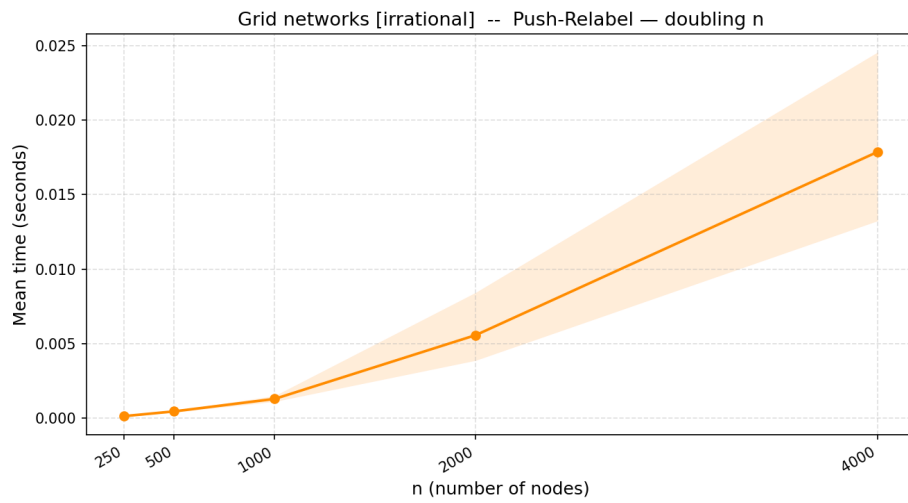


Figure 7: Push-Relabel — Grid, capacità irrazionali, $d = 5$

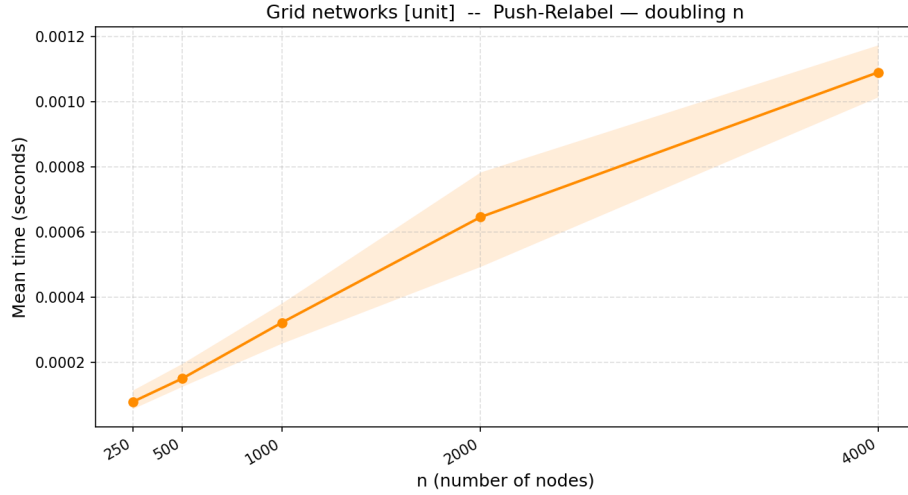


Figure 8: Push-Relabel — Grid, capacità unitarie, $d = 5$

8.4.3 Grafi a layer

n Numero totale di nodi del grafo. Determina implicitamente anche la struttura a griglia (L , W).

d Grado medio in uscita per nodo. Controlla la densità degli archi sia intra-livello ($\lfloor d/2 \rfloor$ archi) sia inter-livello (d archi verso il livello successivo). *Non* rappresenta il numero di livelli, nonostante il nome possa suggerirlo.

Parametri derivati (non impostati direttamente):

L Numero di livelli, calcolato come $2\sqrt{n/2}$.

W Larghezza di ciascun livello, calcolata come $\sqrt{n/2}$.

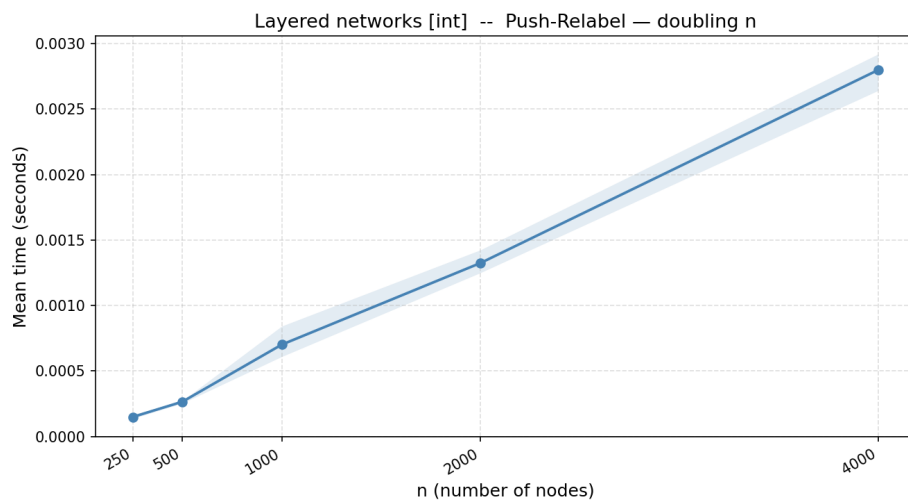


Figure 9: PR — Layered, int, $d = 6$

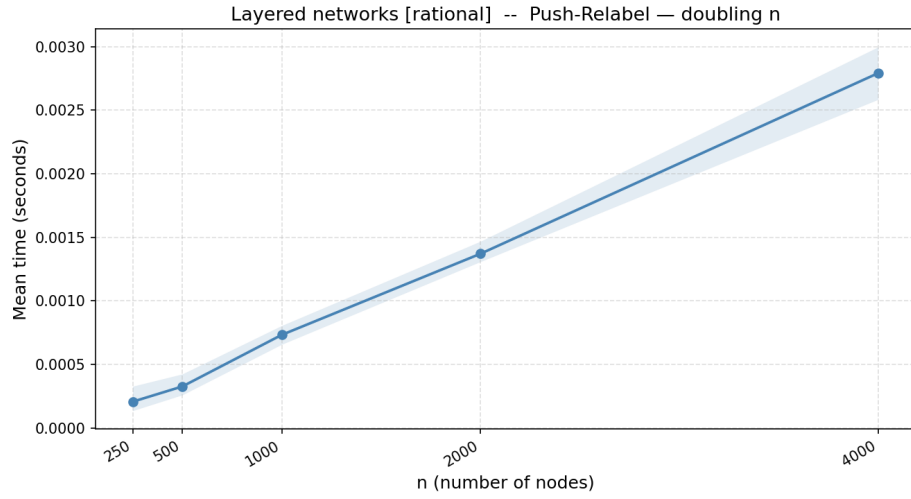


Figure 10: PR — Layered, rat, $d = 6$

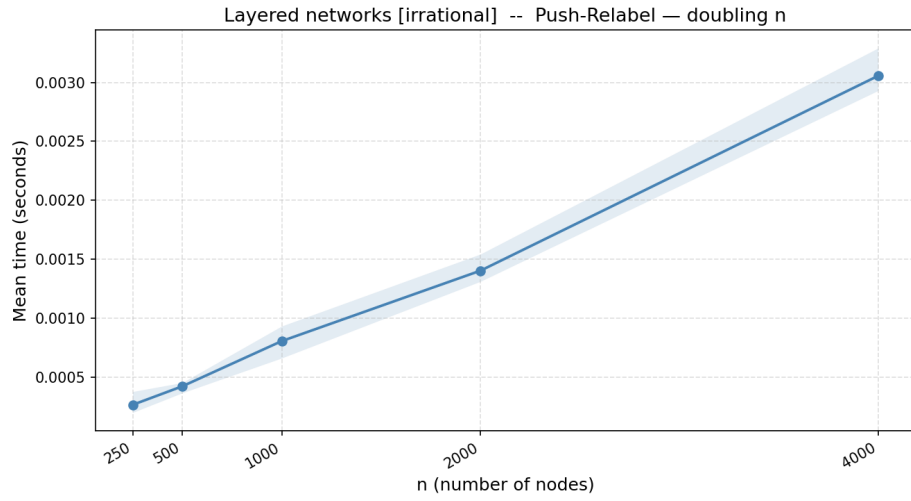


Figure 11: PR — Layered, irrat, $d = 6$

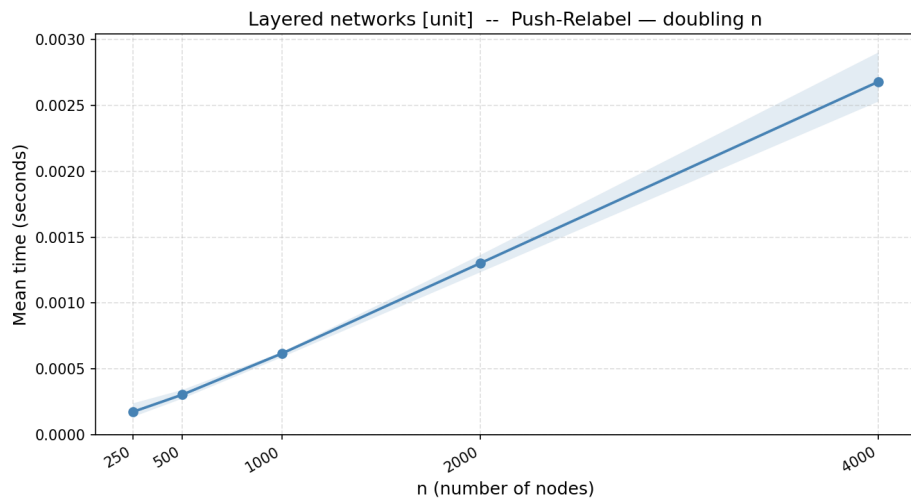


Figure 12: PR — Layered, unit, $d = 6$

8.5 Capacity Scaling

8.5.1 Grafi Erdős–Rényi

n Numero totale di nodi.

$d = p * 100$ Probabilità di inserimento di ciascun arco $u \rightarrow v$ (con $u < v$). È il parametro effettivamente usato dalla funzione di generazione, qui andiamo ad aumentarlo per influenzare solo il limite di archi atteso e vedere la sua relazione con la complessità dell'algoritmo.

hi aumento del massimo della capacità nel generatore per influenzare la componenete $\log U$ della complessità dell'algoritmo

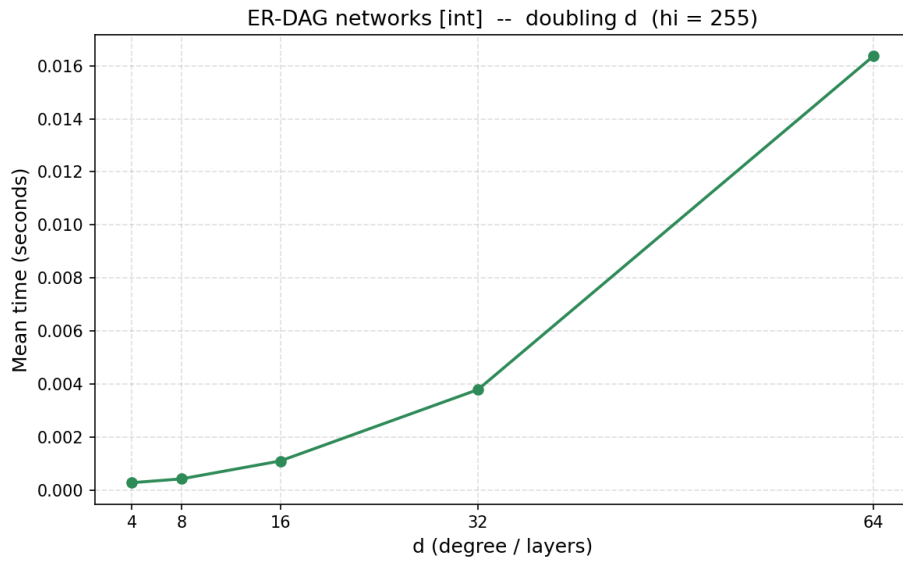


Figure 13: Capacity Scaling — Erdag, capacità intere, $hi = 255$

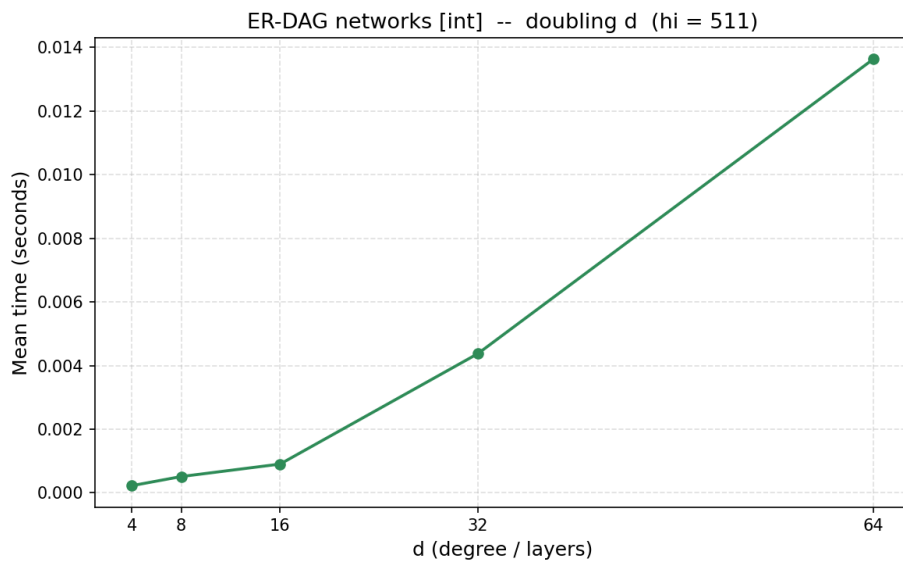


Figure 14: Capacity Scaling — Erdag, capacità intere, $hi = 511$

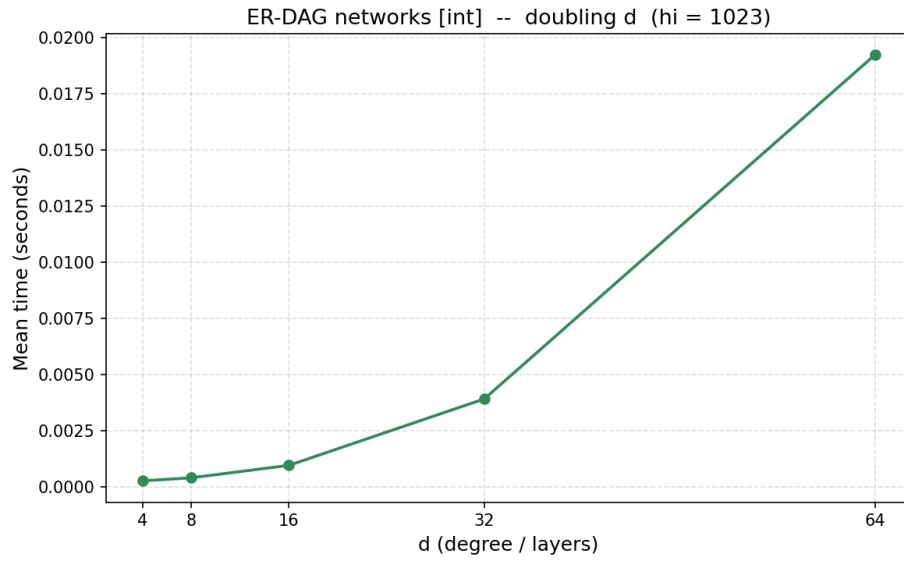


Figure 15: Capacity Scaling — Erdag, capacità intere, $hi = 1023$

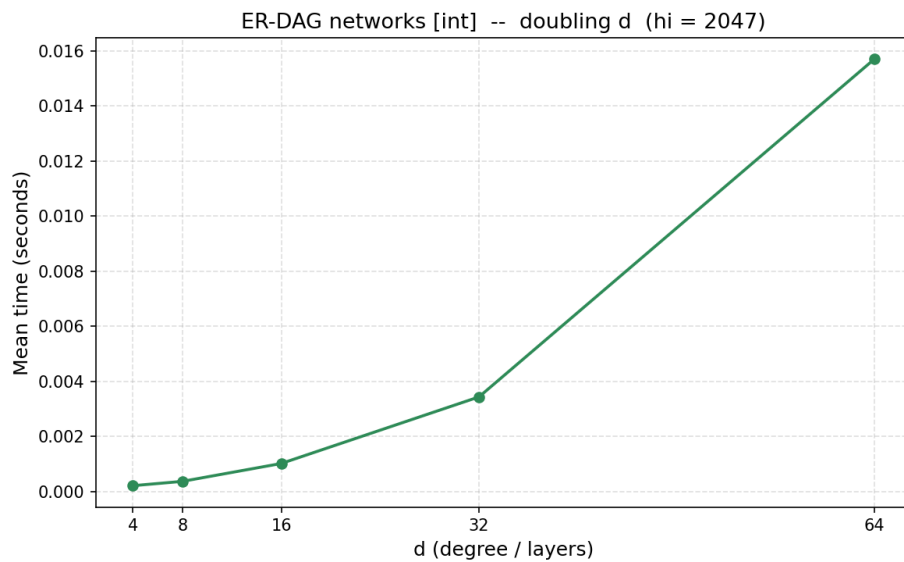


Figure 16: Capacity Scaling — Erdag, capacità intere, $hi = 2047$

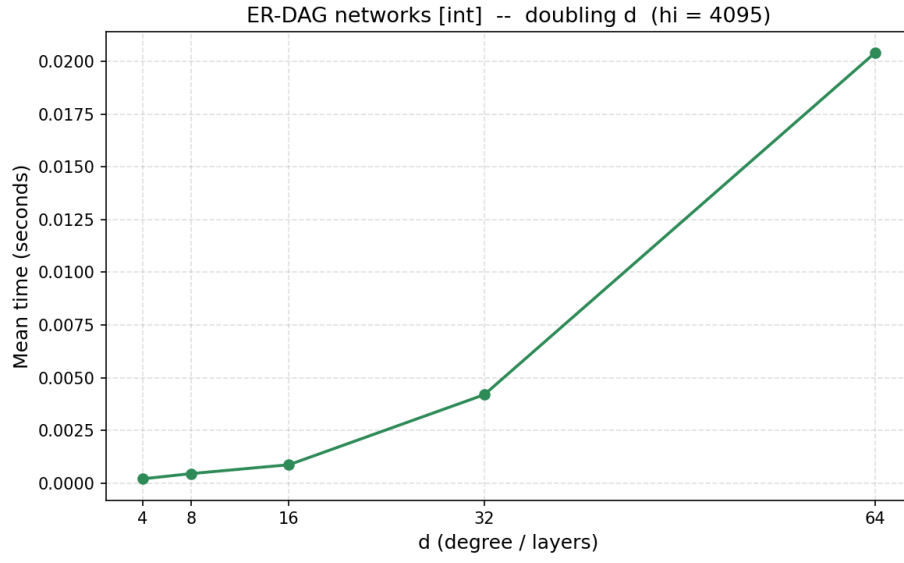


Figure 17: Capacity Scaling — Erdag, capacità intere, $hi = 4095$

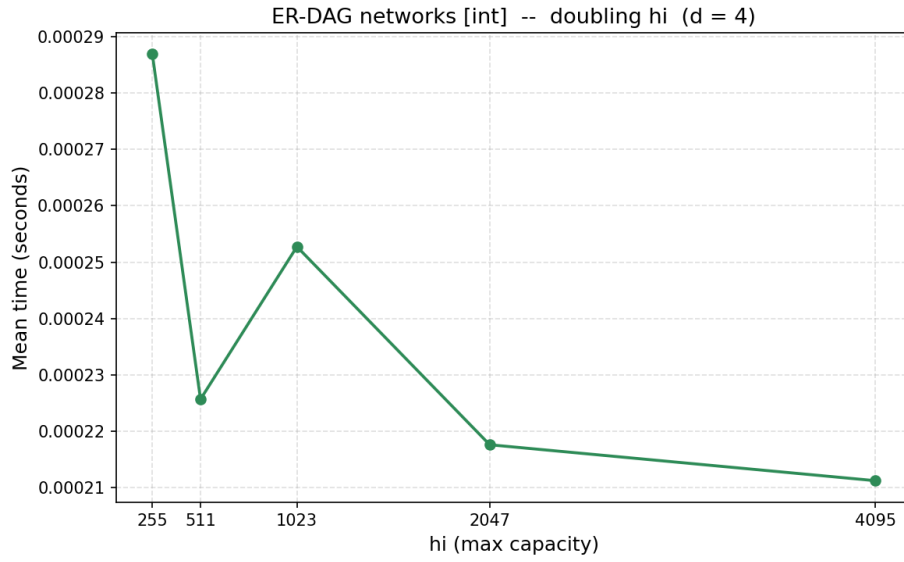


Figure 18: Capacity Scaling — Erdag, capacità intere, $d = 4$

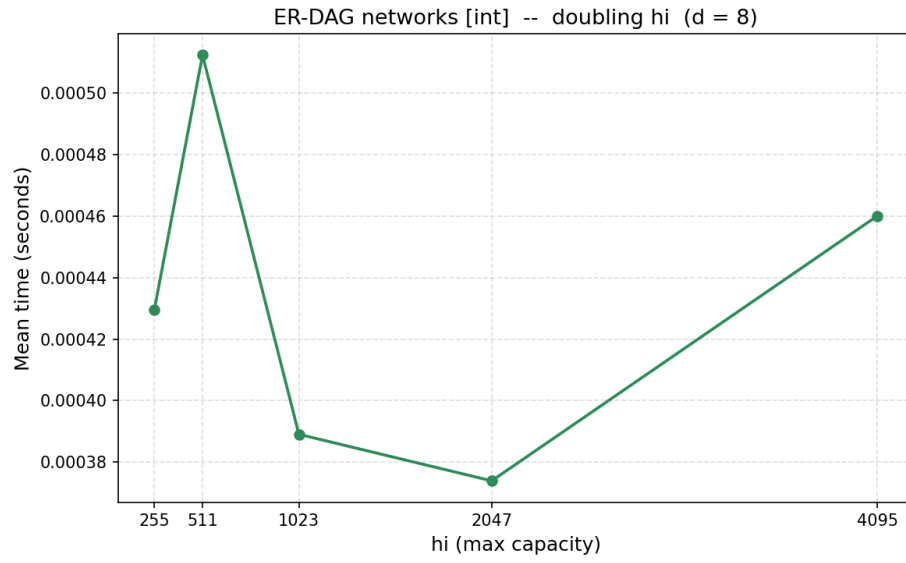


Figure 19: Capacity Scaling — Erdag, capacità intere, $d = 8$

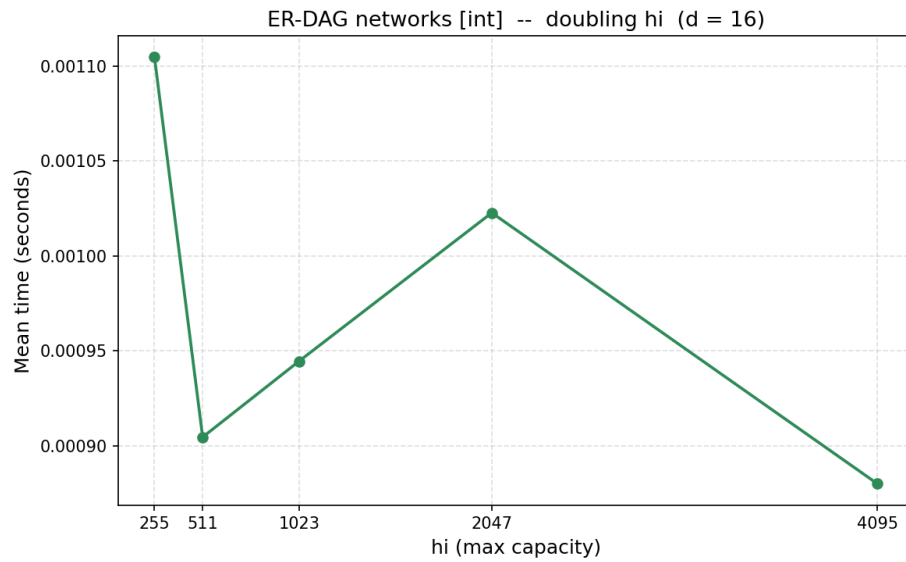


Figure 20: Capacity Scaling — Erdag, capacità intere, $d = 16$

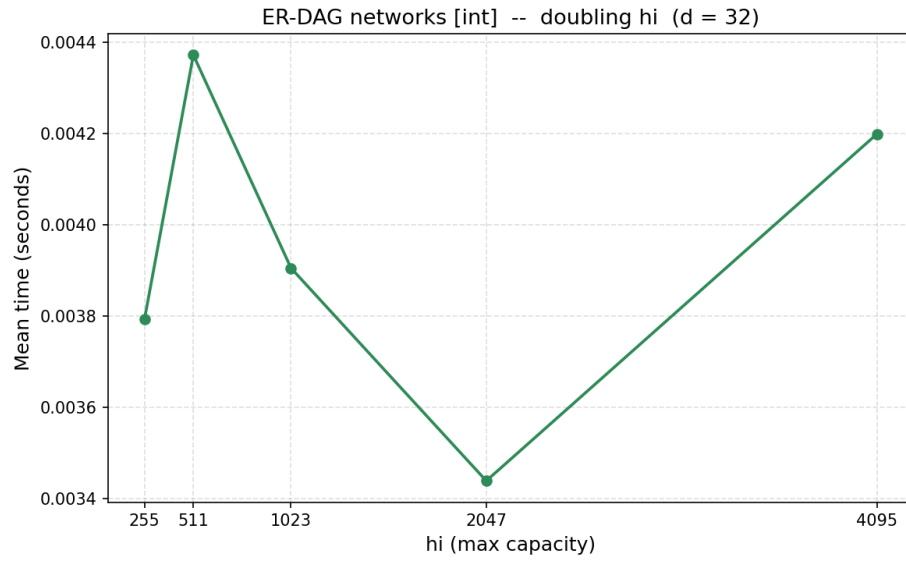


Figure 21: Capacity Scaling — Erdag, capacità intere, $d = 32$

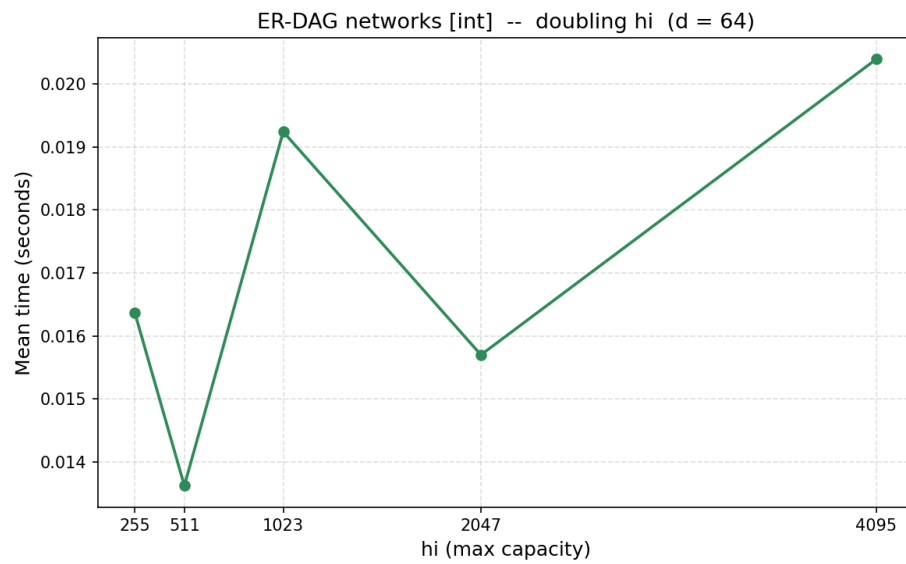


Figure 22: Capacity Scaling — Erdag, capacità intere, $d = 64$

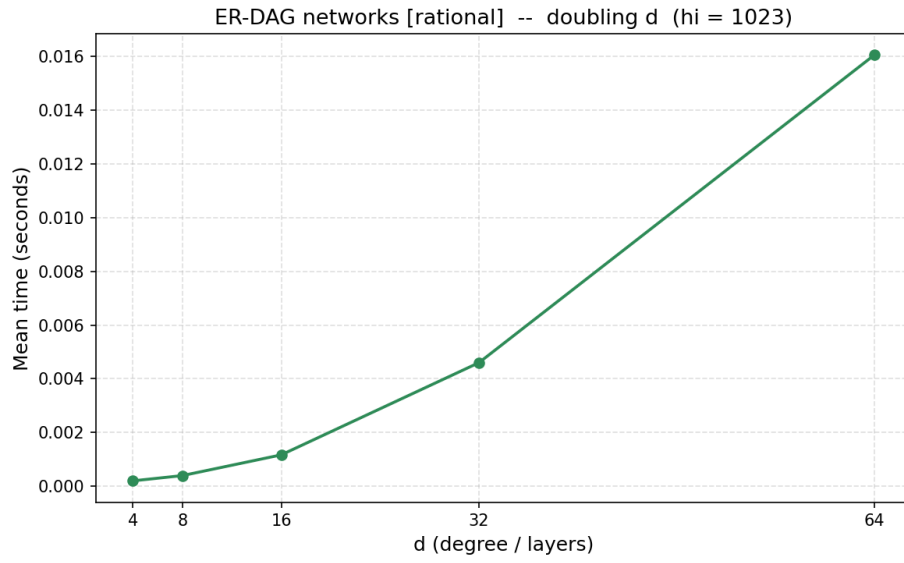


Figure 23: Capacity Scaling — Erdag, capacità frazionarie, $hi = 1023$

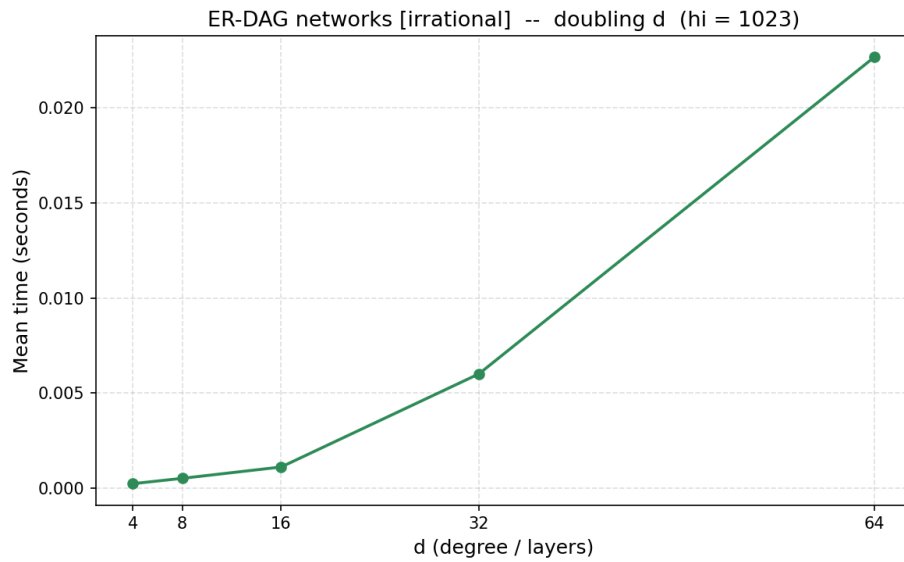


Figure 24: Capacity Scaling — Erdag, capacità irrazionali, $hi = 1023$

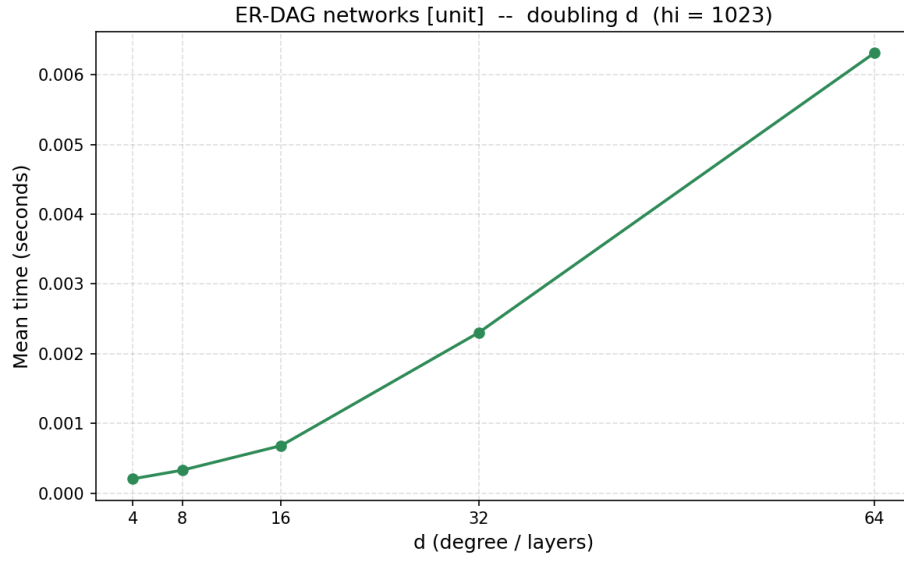


Figure 25: Capacity Scaling — Erdag, capacità intere, $hi = 1023$

8.5.2 Grafi a griglia bidimensionale

n Numero totale di nodi. Deve essere divisibile per d .

$d = 5$ Numero di righe della griglia. Il numero di colonne è derivato come n/d . A differenza dei layered, d qui ha un significato strutturale diretto e univoco (non è un grado medio).

hi aumento del massimo della capacità nel generatore per influenzare la componente $\log U$ della complessità dell'algoritmo

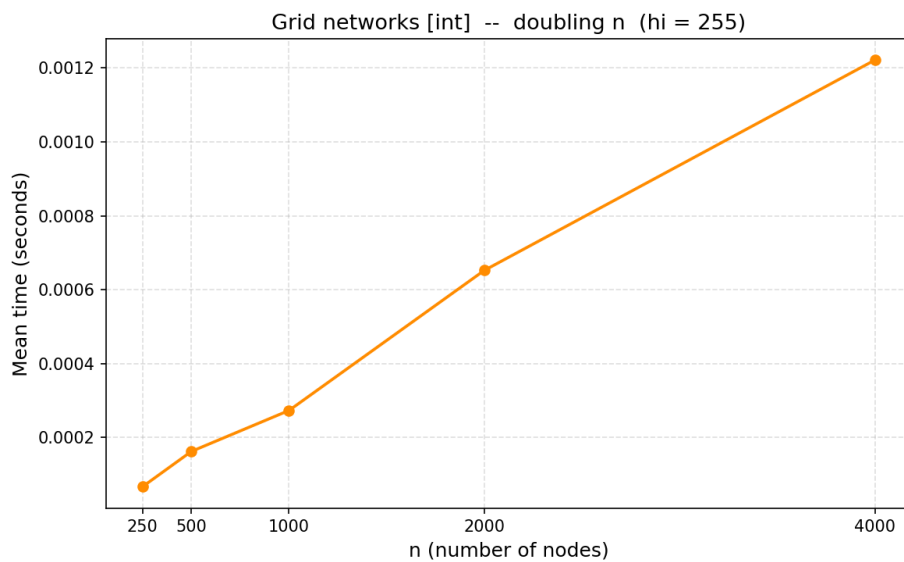


Figure 26: Capacity Scaling — Grid, capacità intere, $hi = 255$

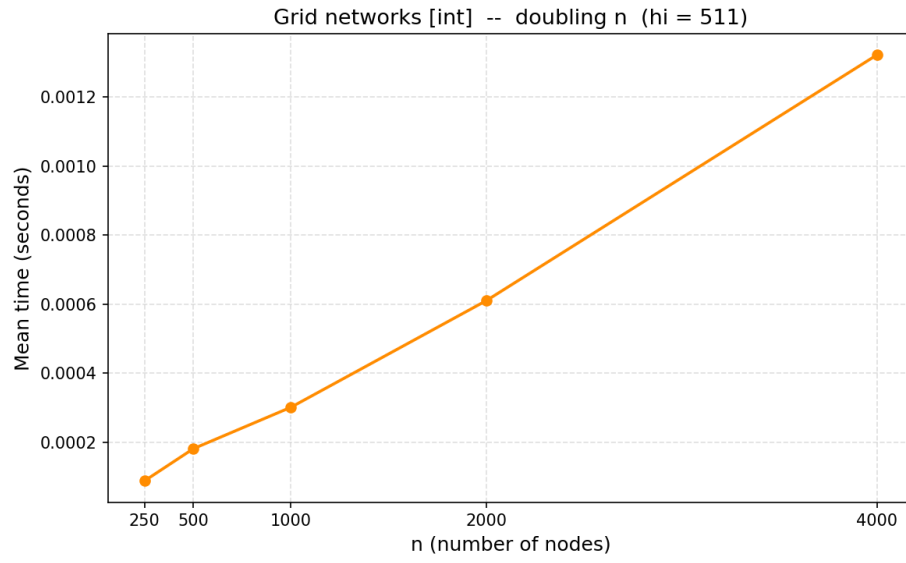


Figure 27: Capacity Scaling — Grid, capacità intere, $hi = 511$

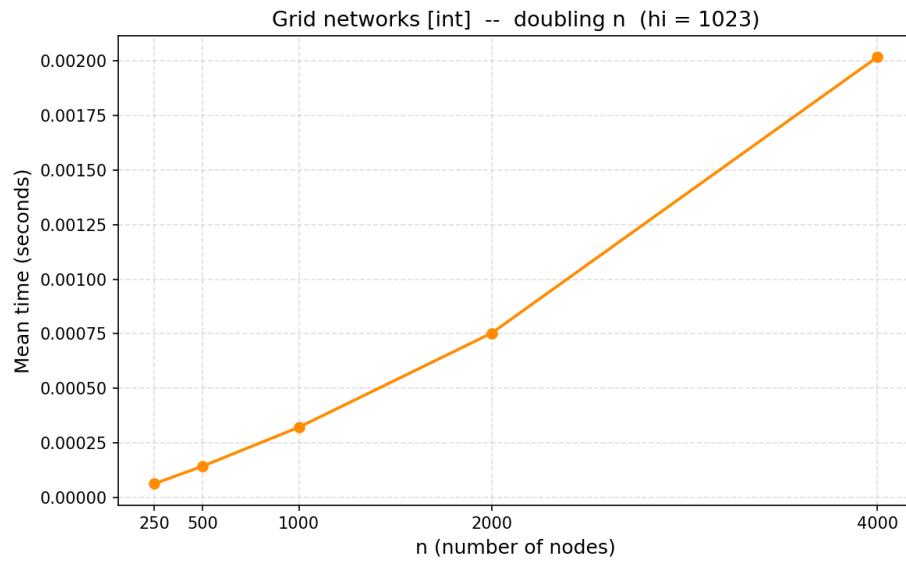


Figure 28: Capacity Scaling — Grid, capacità intere, $hi = 1023$

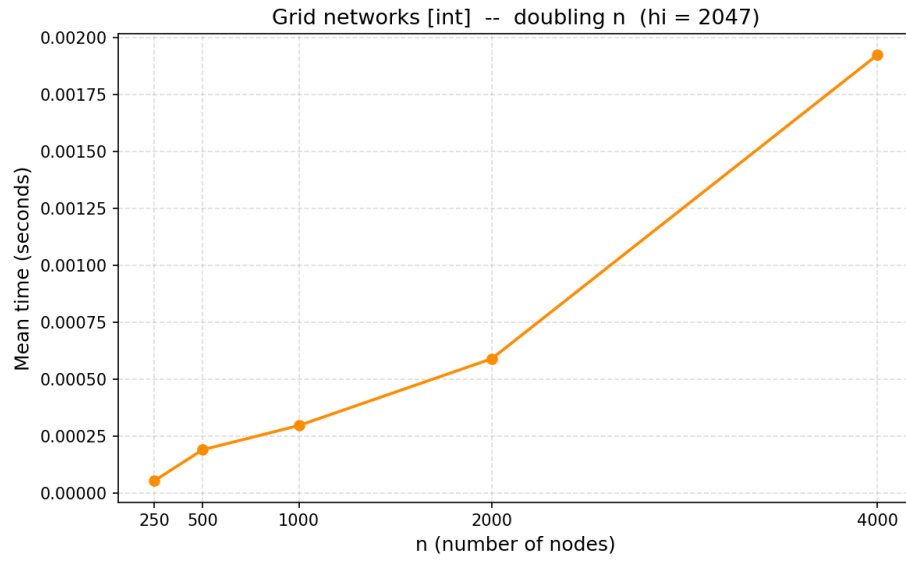


Figure 29: Capacity Scaling — Grid, capacità intere, $hi = 2047$

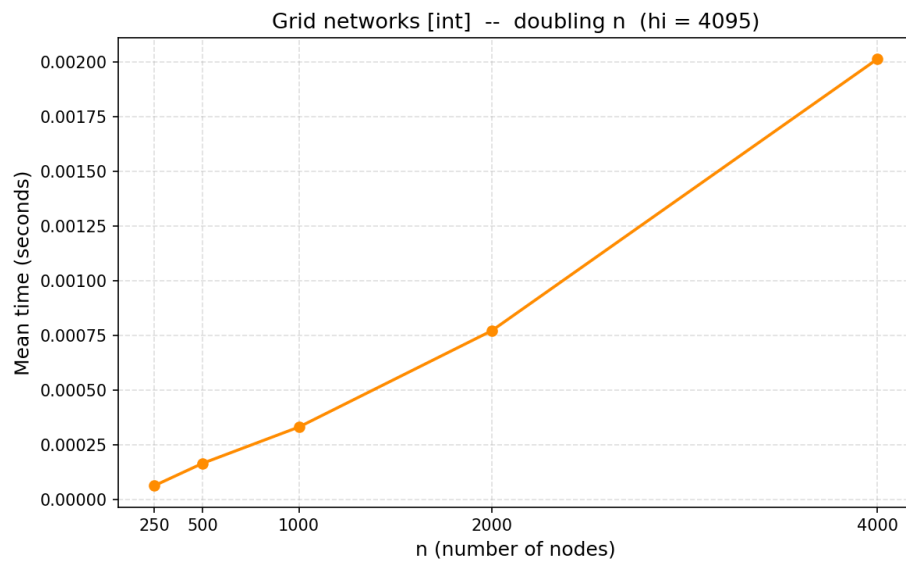
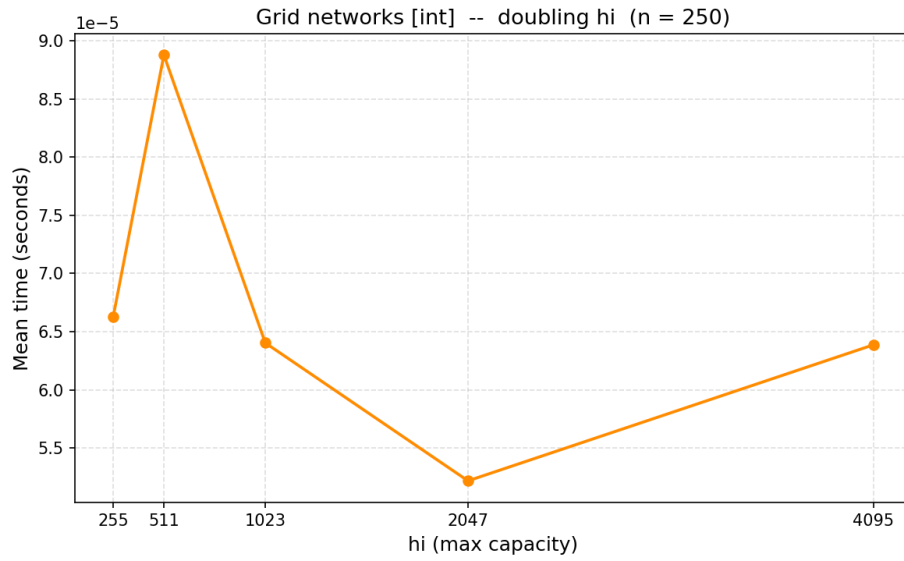
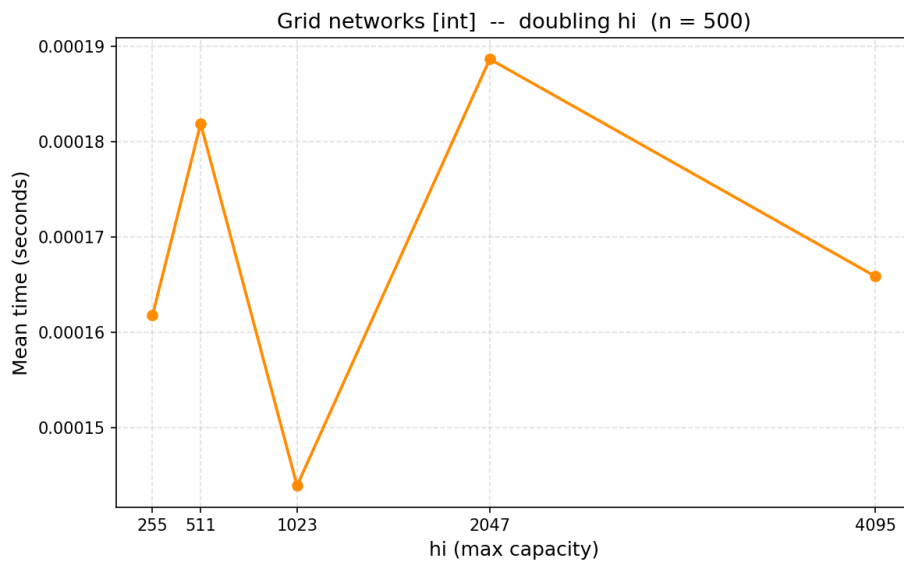


Figure 30: Capacity Scaling — Grid, capacità intere, $hi = 4095$

**Figure 31:** Capacity Scaling — Grid, capacità intere, $n = 250$ **Figure 32:** Capacity Scaling — Grid, capacità intere, $n = 500$

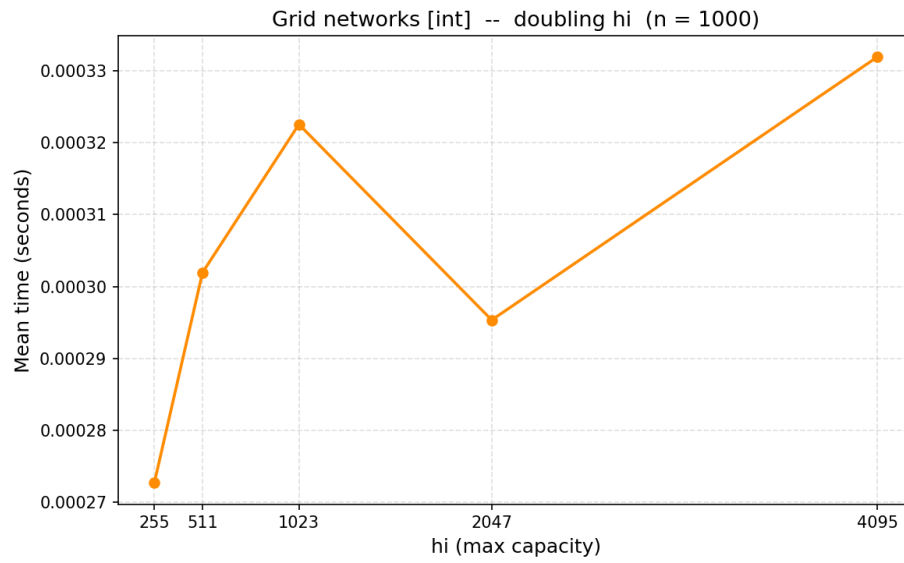


Figure 33: Capacity Scaling — Grid, capacità intere, $n = 1000$

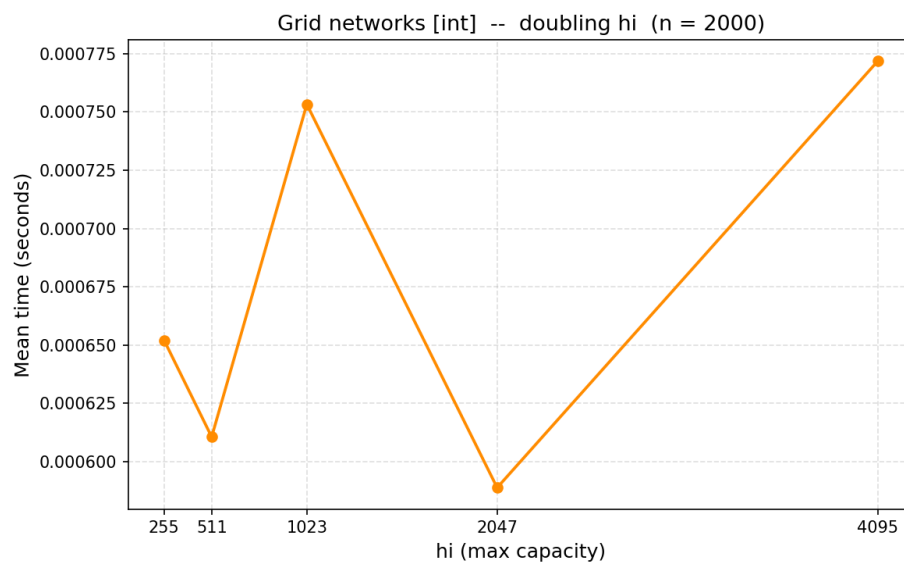


Figure 34: Capacity Scaling — Grid, capacità intere, $n = 2000$

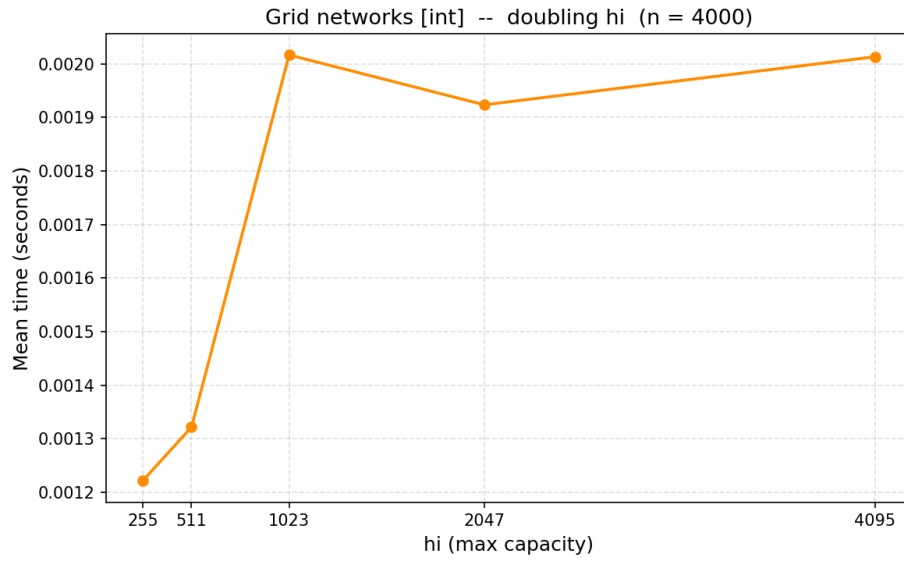


Figure 35: Capacity Scaling — Grid, capacità intere, $n = 4000$

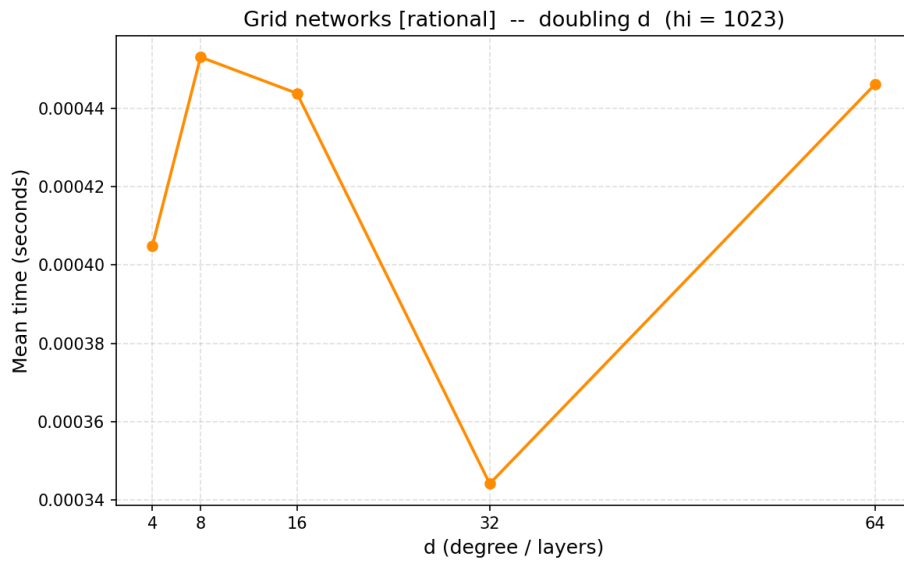


Figure 36: Capacity Scaling — Grid, capacità frazionarie, $h_i = 1023$

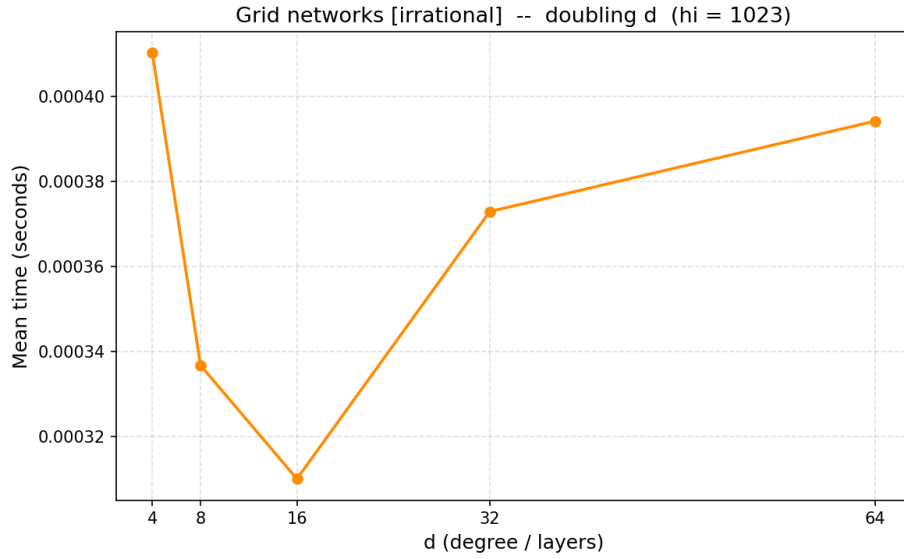


Figure 37: Capacity Scaling — Grid, capacità irrazionali, $hi = 1023$

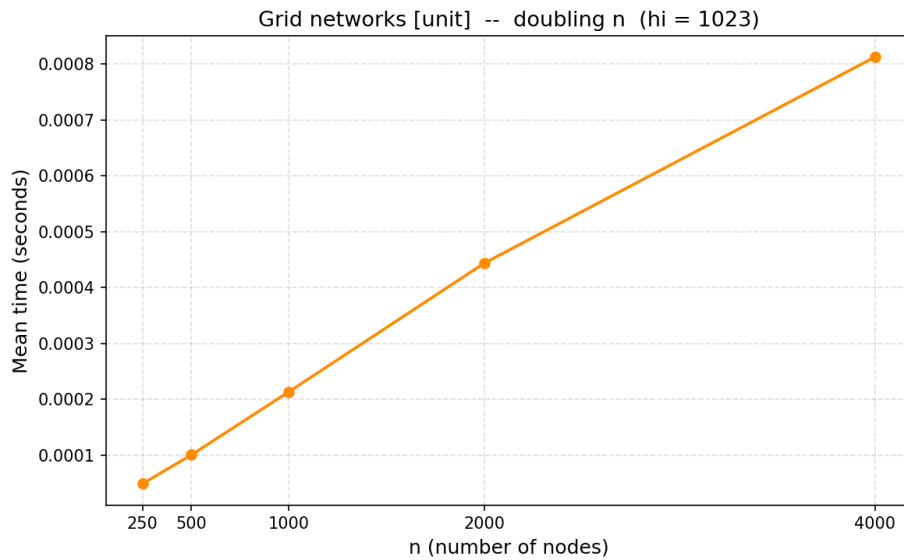


Figure 38: Capacity Scaling — Grid, capacità unitarie, $hi = 1$

8.5.3 Grafo Layered

n Numero totale di nodi del grafo. Determina implicitamente anche la struttura a griglia (L , W).

d Grado medio in uscita per nodo. Controlla la densità degli archi sia intra-livello ($\lfloor d/2 \rfloor$ archi) sia inter-livello (d archi verso il livello successivo). *Non* rappresenta il numero di livelli, nonostante il nome possa suggerirlo.

Parametri derivati (non impostati direttamente):

L Numero di livelli, calcolato come $2\sqrt{n/2}$.

W Larghezza di ciascun livello, calcolata come $\sqrt{n/2}$.

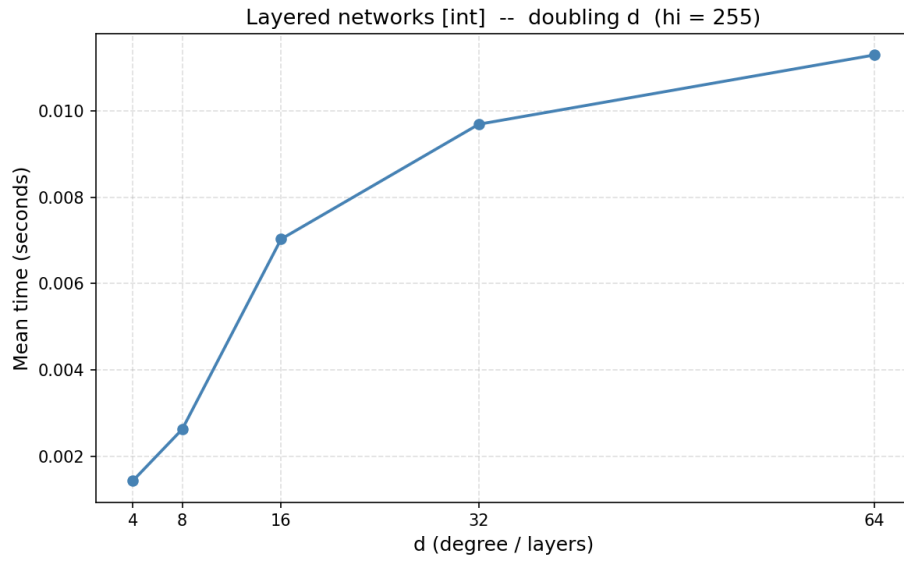


Figure 39: Capacity Scaling — Layer, capacità intere, $hi = 255$

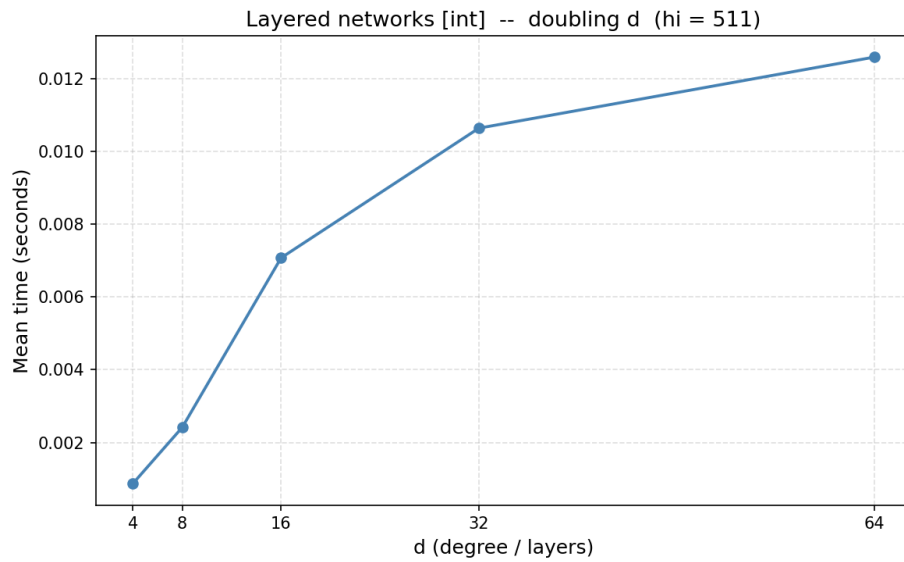


Figure 40: Capacity Scaling — Layer, capacità intere, $hi = 511$

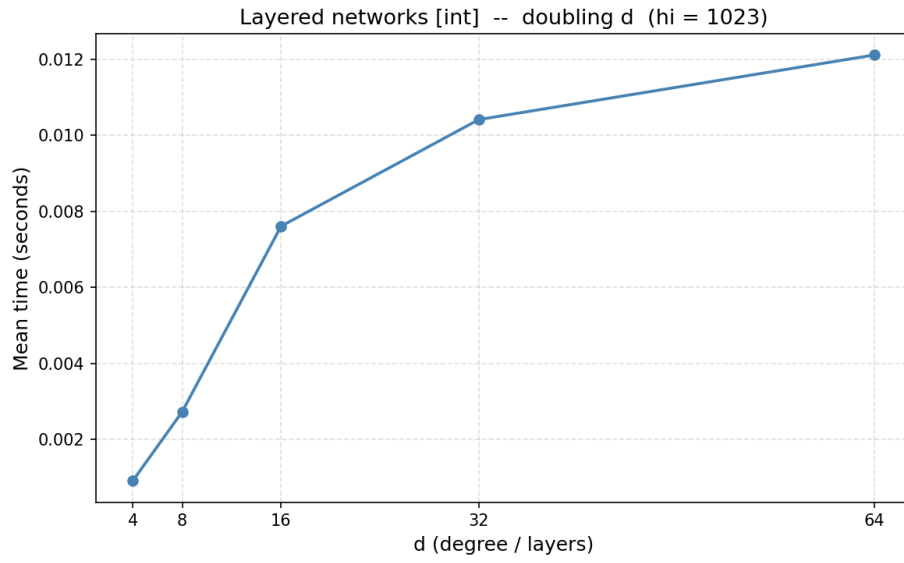


Figure 41: Capacity Scaling — Layer, capacità intere, $hi = 1023$

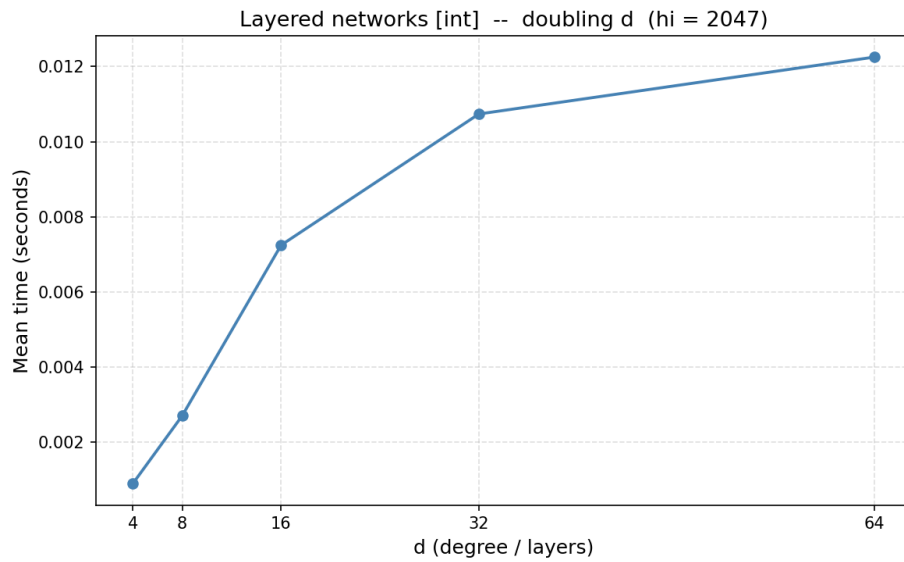


Figure 42: Capacity Scaling — Layer, capacità intere, $hi = 2047$

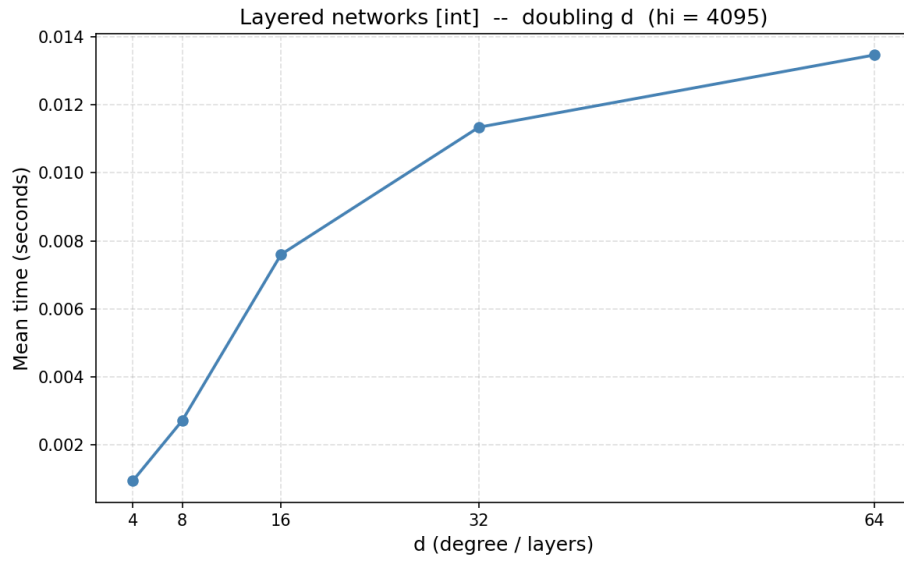


Figure 43: Capacity Scaling — Layer, capacità intere, $hi = 4095$

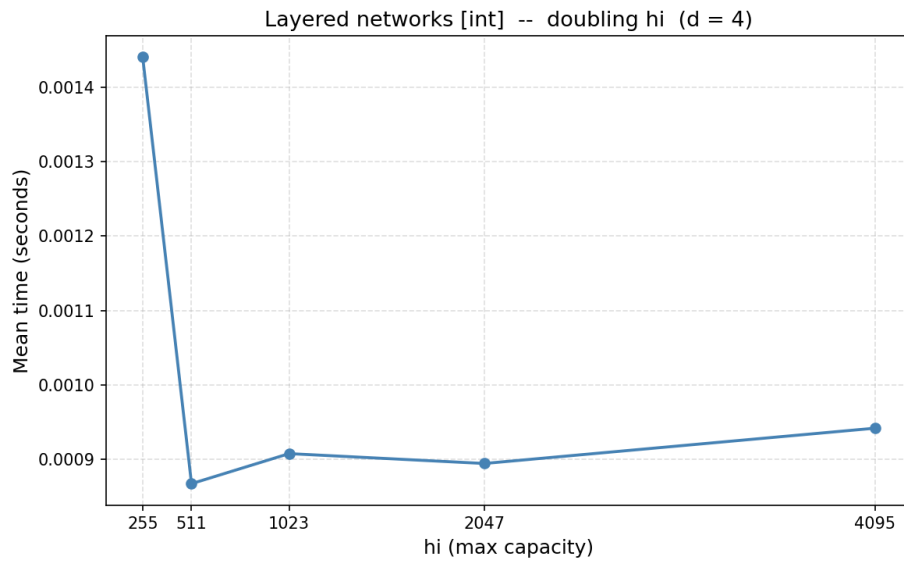


Figure 44: Capacity Scaling — Layer, capacità intere, $d = 4$

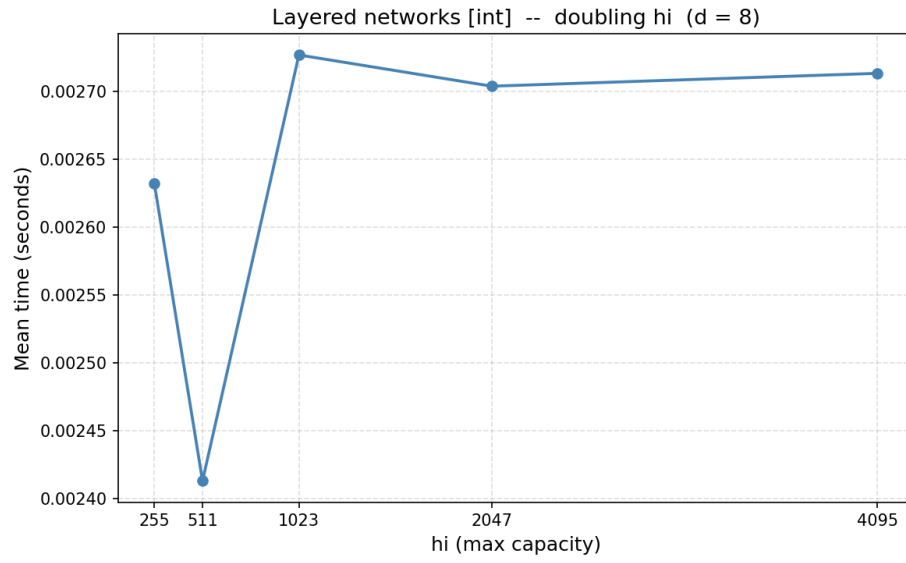


Figure 45: Capacity Scaling — Layer, capacità intere, $d = 8$

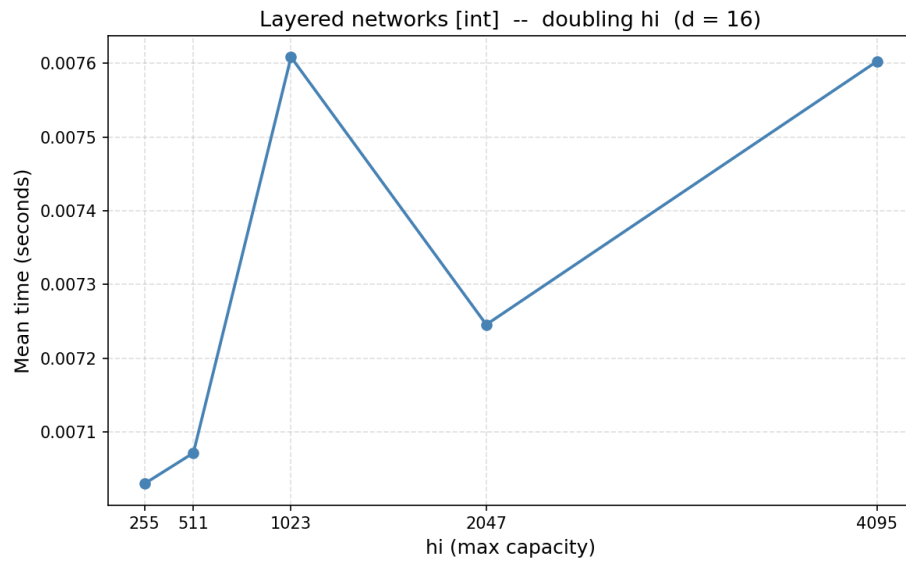


Figure 46: Capacity Scaling — Layer, capacità intere, $d = 16$

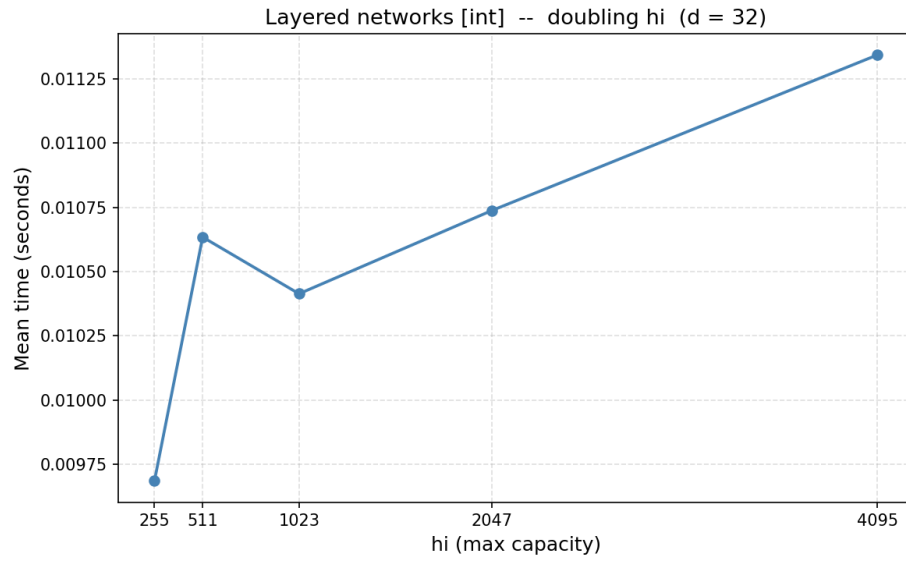


Figure 47: Capacity Scaling — Layer, capacità intere, $d = 32$

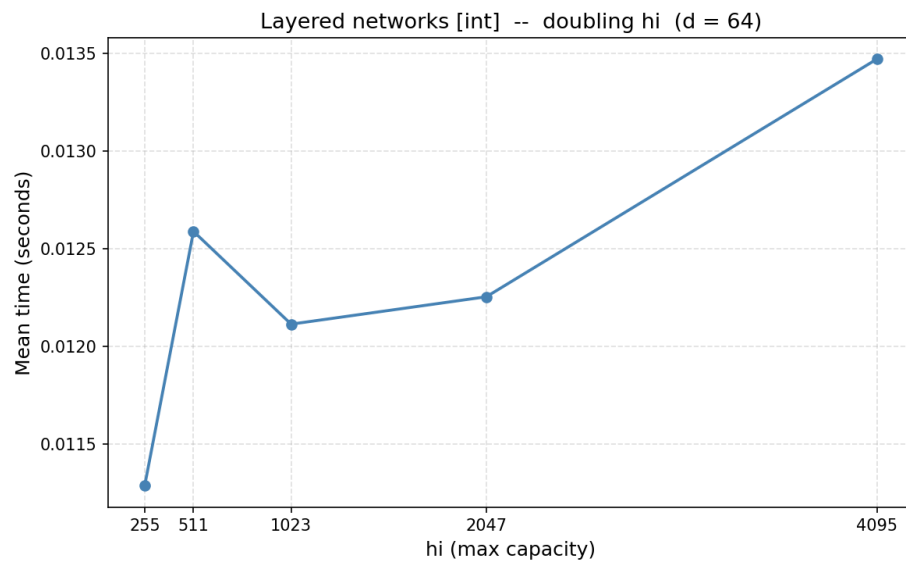


Figure 48: Capacity Scaling — Layer, capacità intere, $d = 64$

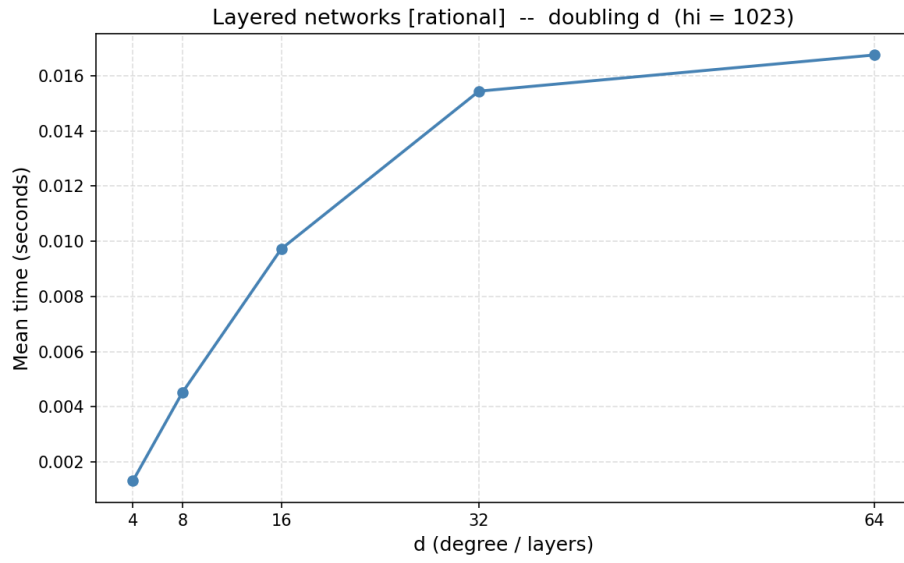


Figure 49: Capacity Scaling — Layer, capacità frazionarie, $hi = 1023$

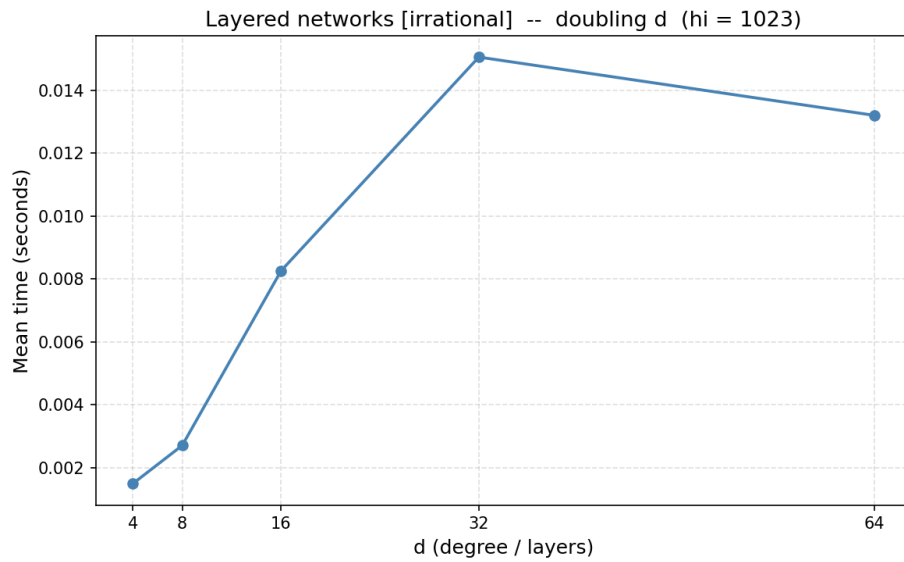


Figure 50: Capacity Scaling — Layer, capacità irrazionali, $hi = 1023$

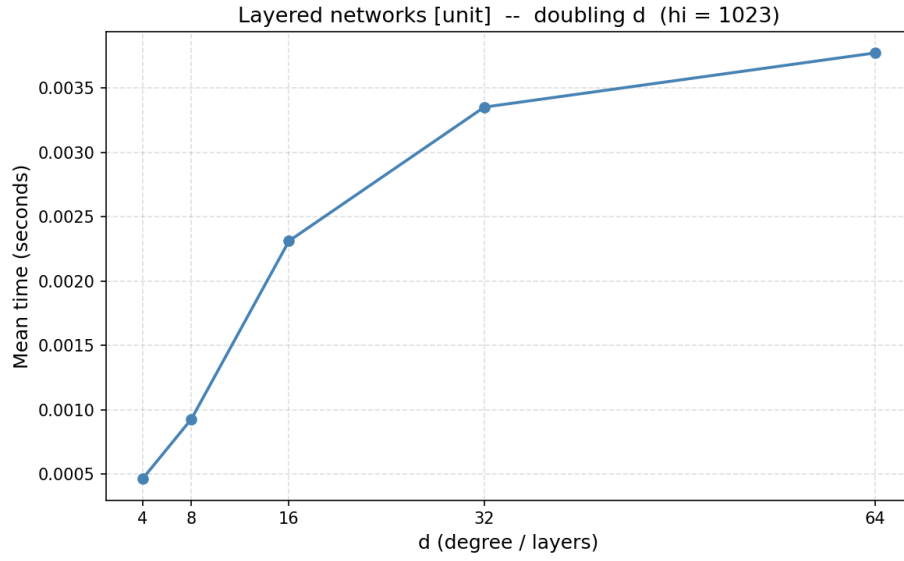


Figure 51: Capacity Scaling — Layer, capacità intere, $hi = 1023$

8.6 Almost-Linear Time

8.6.1 Grafi Erdős–Rényi

n Numero totale di nodi.

p Probabilità di inserimento di ciascun arco $u \rightarrow v$ (con $u < v$). È il parametro effettivamente usato dalla funzione di generazione, qui andiamo ad aumentarlo per influenzare solo il limite di archi atteso e vedere la sua relazione con la complessità dell'algoritmo.

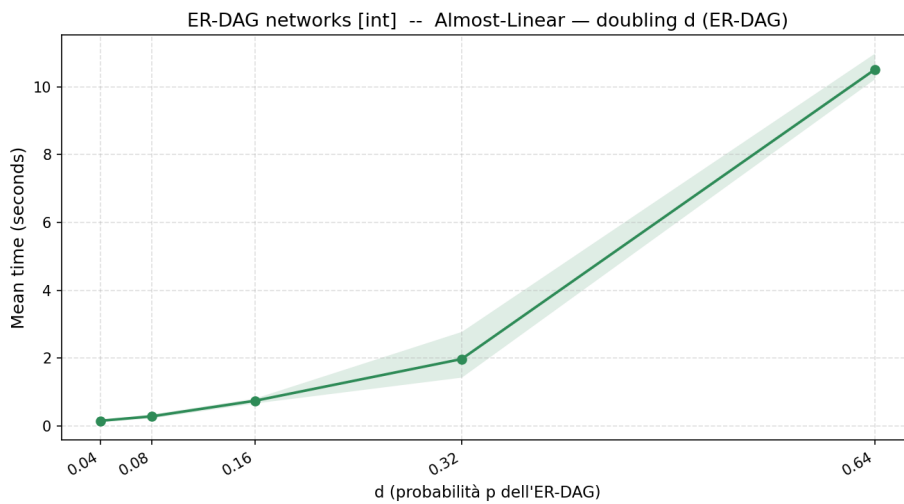


Figure 52: Almost-Linear Time — Erdag, capacità intere, $n = 20$

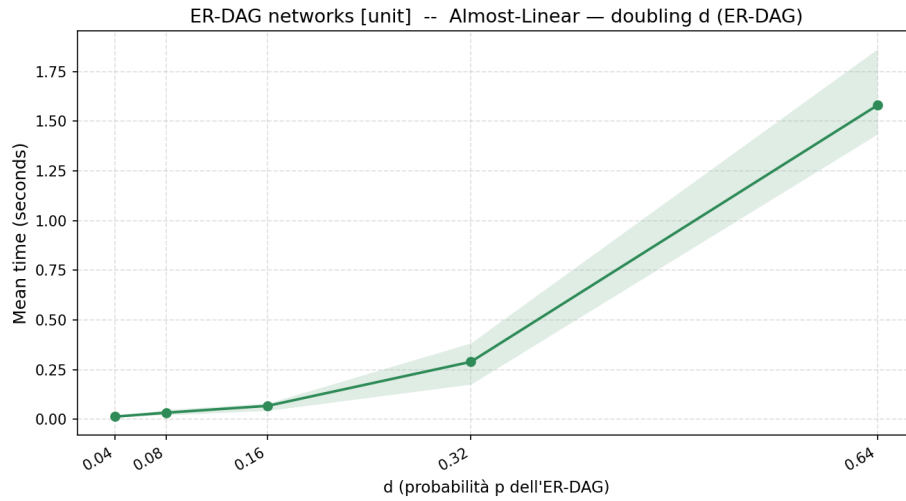


Figure 53: Almost-Linear Time — Erdag, capacità unitarie, $n = 20$

8.6.2 Grafi a griglia bidimensionale

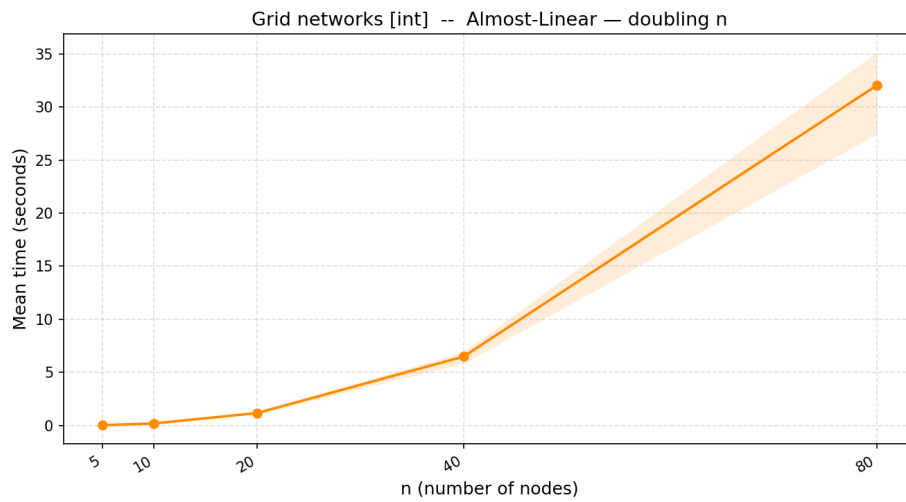


Figure 54: Almost-Linear Time — Grid, capacità intere, $d = 5$

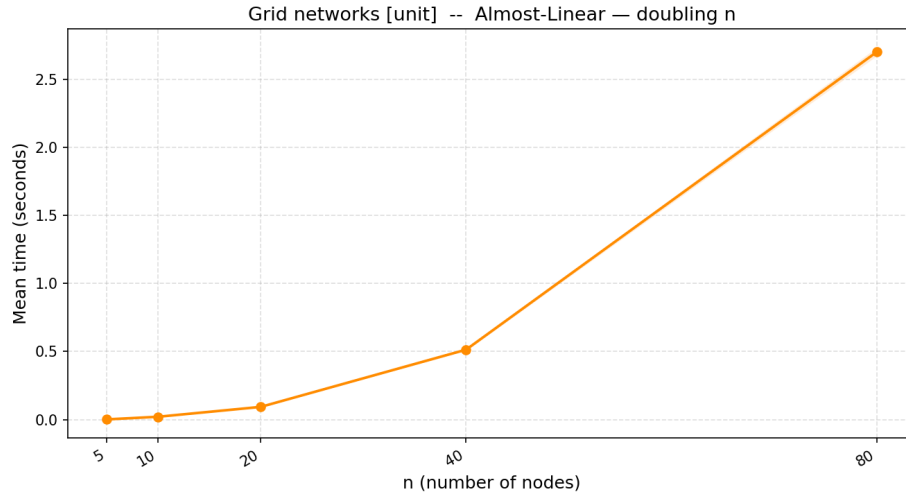


Figure 55: Almost-Linear Time — Grid, capacità unit, $d = 5$

8.6.3 Grafo Layered

n Numero totale di nodi del grafo. Determina implicitamente anche la struttura a griglia (L , W).

d Grado medio in uscita per nodo. Controlla la densità degli archi sia intra-livello ($\lfloor d/2 \rfloor$ archi) sia inter-livello (d archi verso il livello successivo). *Non* rappresenta il numero di livelli, nonostante il nome possa suggerirlo.

Parametri derivati (non impostati direttamente):

L Numero di livelli, calcolato come $2\sqrt{n/2}$.

W Larghezza di ciascun livello, calcolata come $\sqrt{n/2}$.

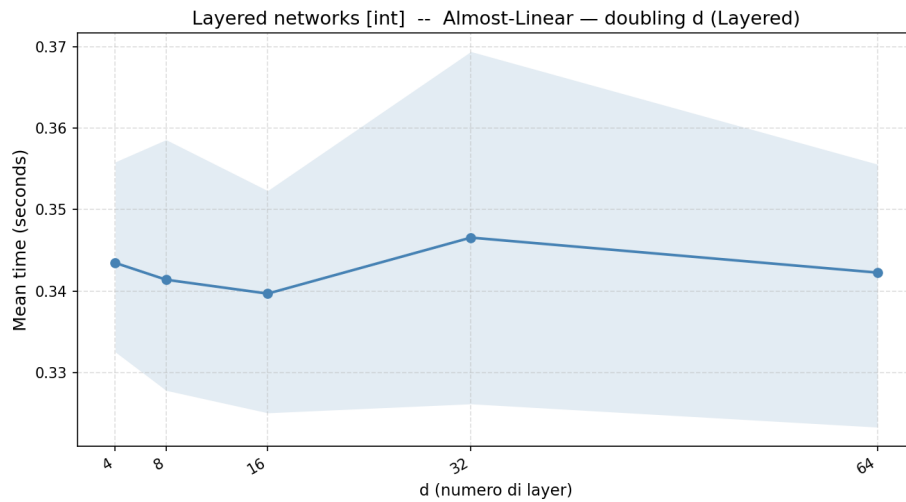


Figure 56: Almost-Linear Time — Layer, capacità intere, $n = 10$

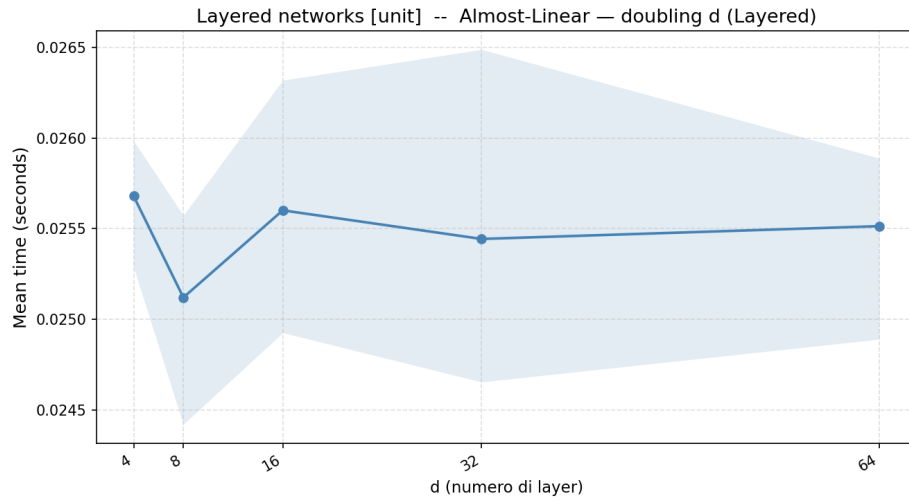


Figure 57: Almost-Linear Time — Layer, capacità unitarie, $n = 10$

9 Discussione

I risultati sperimentali evidenziano come il comportamento pratico degli algoritmi di massimo flusso dipenda fortemente dalla struttura del grafo e dal tipo di capacità.

In particolare:

9.1 Push–Relabel

Risulta generalmente il più competitivo su una vasta gamma di istanze, grazie alla sua capacità di sfruttare efficacemente le euristiche disponibili, la complessità pari a $O(n^3)$ non comporta una crescita di 8x del tempo di esecuzione al raddoppio del numero di vertici nella maggior parte dei casi analizzati soprattutto escludendo gli ERDAG.

Table 6: Risultati aggregati – Push-Relabel (PR), doubling su n (d fisso).

Tipo grafo	Capacità	n	d	Tempo medio (s)	Rapporto
ERDAG	int	250	0.02	0.000067	—
ERDAG	int	500	0.02	0.000329	4.885
ERDAG	int	1000	0.02	0.001053	3.196
ERDAG	int	2000	0.02	0.003330	3.163
ERDAG	int	4000	0.02	0.020478	6.150
ERDAG	rational	250	0.02	0.000099	—
ERDAG	rational	500	0.02	0.000258	2.602
ERDAG	rational	1000	0.02	0.000925	3.587
ERDAG	rational	2000	0.02	0.005130	5.547
ERDAG	rational	4000	0.02	0.023263	4.534
ERDAG	irrational	250	0.02	0.000085	—
ERDAG	irrational	500	0.02	0.000277	3.243
ERDAG	irrational	1000	0.02	0.000891	3.213
ERDAG	irrational	2000	0.02	0.004259	4.781
ERDAG	irrational	4000	0.02	0.033290	7.816
ERDAG	unit	250	0.02	0.000089	—
ERDAG	unit	500	0.02	0.000241	2.692
ERDAG	unit	1000	0.02	0.000825	3.426
ERDAG	unit	2000	0.02	0.003301	4.002
ERDAG	unit	4000	0.02	0.017399	5.270
Grid	int	250	5	0.000116	—
Grid	int	500	5	0.000351	3.030
Grid	int	1000	5	0.001551	4.426
Grid	int	2000	5	0.003646	2.350
Grid	int	4000	5	0.017310	4.748
Grid	rational	250	5	0.000123	—

Tipo grafo	Capacità	n	d	Tempo medio (s)	Rapporto
Grid	rational	500	5	0.000332	2.698
Grid	rational	1000	5	0.001406	4.228
Grid	rational	2000	5	0.004318	3.072
Grid	rational	4000	5	0.019430	4.500
Grid	irrational	250	5	0.000148	—
Grid	irrational	500	5	0.000467	3.151
Grid	irrational	1000	5	0.001304	2.793
Grid	irrational	2000	5	0.005579	4.278
Grid	irrational	4000	5	0.017867	3.202
Grid	unit	250	5	0.000079	—
Grid	unit	500	5	0.000151	1.906
Grid	unit	1000	5	0.000322	2.132
Grid	unit	2000	5	0.000646	2.005
Grid	unit	4000	5	0.001090	1.687
Layered	int	250	6	0.000148	—
Layered	int	500	6	0.000265	1.785
Layered	int	1000	6	0.000701	2.646
Layered	int	2000	6	0.001324	1.889
Layered	int	4000	6	0.002798	2.113
Layered	rational	250	6	0.000206	—
Layered	rational	500	6	0.000326	1.581
Layered	rational	1000	6	0.000732	2.244
Layered	rational	2000	6	0.001371	1.872
Layered	rational	4000	6	0.002791	2.037
Layered	irrational	250	6	0.000265	—
Layered	irrational	500	6	0.000422	1.594
Layered	irrational	1000	6	0.000805	1.909
Layered	irrational	2000	6	0.001403	1.742
Layered	irrational	4000	6	0.003055	2.178
Layered	unit	250	6	0.000173	—
Layered	unit	500	6	0.000304	1.760
Layered	unit	1000	6	0.000616	2.025
Layered	unit	2000	6	0.001303	2.115
Layered	unit	4000	6	0.002680	2.056

9.2 Capacity scaling

Data la complessità $O(m^2 \log U)$ notiamo come il tempo di esecuzione cresca più di 4x al raddoppio del numero di archi in alcuni casi, ma rimanendo comunque dipendente dalla distribuzione delle

capacità del grafo in quando non verificabile per tutte le varianti con capacità massima uguale per gli interi. Per quanto riguarda il termine $\log U$, questo non è correlato da un aumento costante del tempo di esecuzione al raddoppio della capacità massima, ma risulta molto meno rilevante come fattore rispetto alla distribuzione randomica delle capacità del grafo, che ne influenzano maggiormente il tempo di esecuzione.

Table 7: Risultati aggregati – Capacity Scaling (CS), doubling su d (erdag/layered) o n (grid), hi fisso.

Tipo grafo	Capacità	n	d	hi	Tempo medio (s)	Rapporto
ERDAG	int	1000	4	255	0.000252	—
ERDAG	int	1000	8	255	0.000537	2.129
ERDAG	int	1000	16	255	0.001101	2.051
ERDAG	int	1000	32	255	0.004380	3.977
ERDAG	int	1000	64	255	0.019376	4.424
ERDAG	int	1000	4	511	0.000253	—
ERDAG	int	1000	8	511	0.000593	2.342
ERDAG	int	1000	16	511	0.001048	1.766
ERDAG	int	1000	32	511	0.003881	3.704
ERDAG	int	1000	64	511	0.014132	3.642
ERDAG	int	1000	4	1023	0.000296	—
ERDAG	int	1000	8	1023	0.000510	1.723
ERDAG	int	1000	16	1023	0.001246	2.442
ERDAG	int	1000	32	1023	0.003745	3.006
ERDAG	int	1000	64	1023	0.015152	4.045
ERDAG	int	1000	4	2047	0.000239	—
ERDAG	int	1000	8	2047	0.000483	2.023
ERDAG	int	1000	16	2047	0.001306	2.703
ERDAG	int	1000	32	2047	0.005574	4.267
ERDAG	int	1000	64	2047	0.014753	2.647
ERDAG	int	1000	4	4095	0.000299	—
ERDAG	int	1000	8	4095	0.000459	1.536
ERDAG	int	1000	16	4095	0.001300	2.832
ERDAG	int	1000	32	4095	0.003944	3.034
ERDAG	int	1000	64	4095	0.020744	5.259
ERDAG	rational	1000	4	1023	0.000239	—
ERDAG	rational	1000	8	1023	0.000435	1.823
ERDAG	rational	1000	16	1023	0.001272	2.925
ERDAG	rational	1000	32	1023	0.004639	3.645
ERDAG	rational	1000	64	1023	0.017812	3.840
ERDAG	irrational	1000	4	1023	0.000219	—
ERDAG	irrational	1000	8	1023	0.000508	2.314

Tipo grafo	Capacità	n	d	hi	Tempo medio (s)	Rapporto
ERDAG	irrational	1000	16	1023	0.002299	4.529
ERDAG	irrational	1000	32	1023	0.004506	1.960
ERDAG	irrational	1000	64	1023	0.008059	1.788
ERDAG	unit	1000	4	1023	0.000184	—
ERDAG	unit	1000	8	1023	0.000366	1.983
ERDAG	unit	1000	16	1023	0.000661	1.808
ERDAG	unit	1000	32	1023	0.001818	2.751
ERDAG	unit	1000	64	1023	0.006757	3.716
Layered	int	1000	4	255	0.000906	—
Layered	int	1000	8	255	0.002519	2.781
Layered	int	1000	16	255	0.006998	2.778
Layered	int	1000	32	255	0.010707	1.530
Layered	int	1000	64	255	0.010630	0.993
Layered	int	1000	4	511	0.000822	—
Layered	int	1000	8	511	0.002429	2.956
Layered	int	1000	16	511	0.007280	2.998
Layered	int	1000	32	511	0.011048	1.518
Layered	int	1000	64	511	0.011367	1.029
Layered	int	1000	4	1023	0.000925	—
Layered	int	1000	8	1023	0.002560	2.769
Layered	int	1000	16	1023	0.007490	2.926
Layered	int	1000	32	1023	0.012147	1.622
Layered	int	1000	64	1023	0.011860	0.976
Layered	int	1000	4	2047	0.000952	—
Layered	int	1000	8	2047	0.002694	2.830
Layered	int	1000	16	2047	0.007198	2.671
Layered	int	1000	32	2047	0.010753	1.494
Layered	int	1000	64	2047	0.011820	1.099
Layered	int	1000	4	4095	0.001075	—
Layered	int	1000	8	4095	0.002609	2.428
Layered	int	1000	16	4095	0.007584	2.907
Layered	int	1000	32	4095	0.010750	1.417
Layered	int	1000	64	4095	0.013185	1.227
Layered	rational	1000	4	1023	0.001542	—
Layered	rational	1000	8	1023	0.004709	3.054
Layered	rational	1000	16	1023	0.009335	1.982
Layered	rational	1000	32	1023	0.016032	1.717
Layered	rational	1000	64	1023	0.017170	1.071
Layered	irrational	1000	4	1023	0.001290	—

Tipo grafo	Capacità	n	d	hi	Tempo medio (s)	Rapporto
Layered	irrational	1000	8	1023	0.002437	1.889
Layered	irrational	1000	16	1023	0.008110	3.328
Layered	irrational	1000	32	1023	0.014017	1.728
Layered	irrational	1000	64	1023	0.012580	0.897
Layered	unit	1000	4	1023	0.000588	—
Layered	unit	1000	8	1023	0.001274	2.166
Layered	unit	1000	16	1023	0.002360	1.852
Layered	unit	1000	32	1023	0.003502	1.484
Layered	unit	1000	64	1023	0.003388	0.967
Grid	int	250	5	255	0.000066	—
Grid	int	500	5	255	0.000162	2.440
Grid	int	1000	5	255	0.000273	1.686
Grid	int	2000	5	255	0.000652	2.391
Grid	int	4000	5	255	0.001222	1.874
Grid	int	250	5	511	0.000089	—
Grid	int	500	5	511	0.000182	2.048
Grid	int	1000	5	511	0.000302	1.660
Grid	int	2000	5	511	0.000611	2.022
Grid	int	4000	5	511	0.001322	2.165
Grid	int	250	5	1023	0.000064	—
Grid	int	500	5	1023	0.000144	2.248
Grid	int	1000	5	1023	0.000323	2.241
Grid	int	2000	5	1023	0.000753	2.336
Grid	int	4000	5	1023	0.002017	2.677
Grid	int	250	5	2047	0.000052	—
Grid	int	500	5	2047	0.000189	3.616
Grid	int	1000	5	2047	0.000295	1.566
Grid	int	2000	5	2047	0.000589	1.993
Grid	int	4000	5	2047	0.001924	3.267
Grid	int	250	5	4095	0.000064	—
Grid	int	500	5	4095	0.000166	2.597
Grid	int	1000	5	4095	0.000332	2.001
Grid	int	2000	5	4095	0.000772	2.326
Grid	int	4000	5	4095	0.002013	2.608
Grid	rational	250	5	1023	0.000096	—
Grid	rational	500	5	1023	0.000207	2.161
Grid	rational	1000	5	1023	0.000395	1.912
Grid	rational	2000	5	1023	0.000868	2.195
Grid	rational	4000	5	1023	0.002327	2.682

Tipo grafo	Capacità	n	d	hi	Tempo medio (s)	Rapporto
Grid	irrational	250	5	1023	0.000104	—
Grid	irrational	500	5	1023	0.000242	2.322
Grid	irrational	1000	5	1023	0.000337	1.394
Grid	irrational	2000	5	1023	0.000771	2.288
Grid	irrational	4000	5	1023	0.001688	2.190
Grid	unit	250	5	1023	0.000049	—
Grid	unit	500	5	1023	0.000100	2.062
Grid	unit	1000	5	1023	0.000213	2.131
Grid	unit	2000	5	1023	0.000444	2.079
Grid	unit	4000	5	1023	0.000813	1.832

Table 8: Risultati aggregati – Capacity Scaling (CS), doubling su hi , n/d fissi.

Tipo grafo	Capacità	n	d	hi	Tempo medio (s)	Rapporto
ERDAG	int	1000	4	255	0.000252	—
ERDAG	int	1000	4	511	0.000253	1.004
ERDAG	int	1000	4	1023	0.000296	1.169
ERDAG	int	1000	4	2047	0.000239	0.807
ERDAG	int	1000	4	4095	0.000299	1.251
ERDAG	int	1000	8	255	0.000537	—
ERDAG	int	1000	8	511	0.000593	1.105
ERDAG	int	1000	8	1023	0.000510	0.860
ERDAG	int	1000	8	2047	0.000483	0.947
ERDAG	int	1000	8	4095	0.000459	0.950
ERDAG	int	1000	16	255	0.001101	—
ERDAG	int	1000	16	511	0.001048	0.951
ERDAG	int	1000	16	1023	0.001246	1.189
ERDAG	int	1000	16	2047	0.001306	1.048
ERDAG	int	1000	16	4095	0.001300	0.995
ERDAG	int	1000	32	255	0.004380	—
ERDAG	int	1000	32	511	0.003881	0.886
ERDAG	int	1000	32	1023	0.003745	0.965
ERDAG	int	1000	32	2047	0.005574	1.488
ERDAG	int	1000	32	4095	0.003944	0.708
ERDAG	int	1000	64	255	0.019376	—
ERDAG	int	1000	64	511	0.014132	0.729
ERDAG	int	1000	64	1023	0.015152	1.072
ERDAG	int	1000	64	2047	0.014753	0.974

Tipo grafo	Capacità	n	d	hi	Tempo medio (s)	Rapporto
ERDAG	int	1000	64	4095	0.020744	1.406
Layered	int	1000	4	255	0.000906	—
Layered	int	1000	4	511	0.000822	0.907
Layered	int	1000	4	1023	0.000925	1.125
Layered	int	1000	4	2047	0.000952	1.030
Layered	int	1000	4	4095	0.001075	1.129
Layered	int	1000	8	255	0.002519	—
Layered	int	1000	8	511	0.002429	0.964
Layered	int	1000	8	1023	0.002560	1.054
Layered	int	1000	8	2047	0.002694	1.053
Layered	int	1000	8	4095	0.002609	0.968
Layered	int	1000	16	255	0.006998	—
Layered	int	1000	16	511	0.007280	1.040
Layered	int	1000	16	1023	0.007490	1.029
Layered	int	1000	16	2047	0.007198	0.961
Layered	int	1000	16	4095	0.007584	1.054
Layered	int	1000	32	255	0.010707	—
Layered	int	1000	32	511	0.011048	1.032
Layered	int	1000	32	1023	0.012147	1.099
Layered	int	1000	32	2047	0.010753	0.885
Layered	int	1000	32	4095	0.010750	1.000
Layered	int	1000	64	255	0.010630	—
Layered	int	1000	64	511	0.011367	1.069
Layered	int	1000	64	1023	0.011860	1.043
Layered	int	1000	64	2047	0.011820	0.997
Layered	int	1000	64	4095	0.013185	1.116
Grid	int	250	5	255	0.000066	—
Grid	int	250	5	511	0.000089	1.339
Grid	int	250	5	1023	0.000064	0.721
Grid	int	250	5	2047	0.000052	0.815
Grid	int	250	5	4095	0.000064	1.224
Grid	int	500	5	255	0.000162	—
Grid	int	500	5	511	0.000182	1.124
Grid	int	500	5	1023	0.000144	0.791
Grid	int	500	5	2047	0.000189	1.311
Grid	int	500	5	4095	0.000166	0.879
Grid	int	1000	5	255	0.000273	—
Grid	int	1000	5	511	0.000302	1.107
Grid	int	1000	5	1023	0.000323	1.068

Tipo grafo	Capacità	n	d	hi	Tempo medio (s)	Rapporto
Grid	int	1000	5	2047	0.000295	0.916
Grid	int	1000	5	4095	0.000332	1.124
Grid	int	2000	5	255	0.000652	—
Grid	int	2000	5	511	0.000611	0.936
Grid	int	2000	5	1023	0.000753	1.234
Grid	int	2000	5	2047	0.000589	0.781
Grid	int	2000	5	4095	0.000772	1.311
Grid	int	4000	5	255	0.001222	—
Grid	int	4000	5	511	0.001322	1.082
Grid	int	4000	5	1023	0.002017	1.526
Grid	int	4000	5	2047	0.001924	0.954
Grid	int	4000	5	4095	0.002013	1.047

9.3 algoritmi quasi-lineari

Pur essendo teoricamente molto efficienti, richiedono strutture dati avanzate e solutori numerici complessi, rendendoli difficili da applicare in contesti pratici e meno competitivi su istanze di dimensione moderata, vediamo infatti come l'aumento del tempo di esecuzione al raddoppio del numero di archi sia ben oltre che lineare, questo però è solo legato all'implementazione adottata senza le strutture dati dell'algoritmo originale, che in assenza di miglioramenti significativi rischierebbero di non rendere l'algoritmo utilizzabile nella pratica. L'implementazione dell'algoritmo è basata su quella in python della tesi[5]

Table 9: Risultati aggregati – Augmenting Layers (AL). Doubling su d per erdag/layered (n fisso), su n per grid (d fisso).

Tipo grafo	Capacità	n	d	Tempo medio (s)	Rapporto
ERDAG	int	20	0.04	0.156647	—
ERDAG	int	20	0.08	0.288358	1.841
ERDAG	int	20	0.16	0.748086	2.594
ERDAG	int	20	0.32	1.976042	2.641
ERDAG	int	20	0.64	10.503263	5.315
ERDAG	unit	20	0.04	0.013166	—
ERDAG	unit	20	0.08	0.032613	2.477
ERDAG	unit	20	0.16	0.067257	2.062
ERDAG	unit	20	0.32	0.288857	4.295
ERDAG	unit	20	0.64	1.579952	5.470
Layered	int	10	4	0.343478	—
Layered	int	10	8	0.341417	0.994
Layered	int	10	16	0.339711	0.995

Tipo grafo	Capacità	n	d	Tempo medio (s)	Rapporto
Layered	int	10	32	0.346578	1.020
Layered	int	10	64	0.342289	0.988
Layered	unit	10	4	0.025679	—
Layered	unit	10	8	0.025120	0.978
Layered	unit	10	16	0.025601	1.019
Layered	unit	10	32	0.025443	0.994
Layered	unit	10	64	0.025514	1.003
Grid	int	5	5	0.033427	—
Grid	int	10	5	0.195882	5.860
Grid	int	20	5	1.175805	6.003
Grid	int	40	5	6.495812	5.525
Grid	int	80	5	32.027383	4.930
Grid	unit	5	5	0.001257	—
Grid	unit	10	5	0.020612	16.396
Grid	unit	20	5	0.093075	4.516
Grid	unit	40	5	0.513595	5.518
Grid	unit	80	5	2.701556	5.260

10 Conclusioni

Il presente studio ha analizzato il comportamento pratico di diversi algoritmi per il calcolo del massimo flusso, confrontandone prestazioni, robustezza e sensibilità alle caratteristiche delle istanze.

I risultati mostrano che:

- push relabel è un algoritmo universalmente migliore, rappresenta spesso la scelta più solida in contesti generali;
- le performance degli algoritmi dipendono dalla struttura del grafo e dal tipo di capacità;
- L'algoritmo Capacity Scaling non risulta influenzato dal componente $\log U$ in quanto questo è meno rilevante rispetto alla struttura o alla distribuzione delle capacità;
- gli algoritmi quasi-lineari, pur interessanti dal punto di vista teorico, presentano limitazioni pratiche significative a causa delle costanti che rendono i tempi di esecuzione relativamente grandi anche per input di piccole dimensioni.

Il lavoro evidenzia l'importanza di un approccio sperimentale rigoroso per valutare le prestazioni reali degli algoritmi di massimo flusso e suggerisce possibili estensioni future, tra cui:

- analisi su istanze di dimensione ancora maggiore;
- studio dell'impatto di tecniche di parallelizzazione;
- integrazione di nuove euristiche per migliorare la scalabilità.

11 Repository del codice

Il codice utilizzato per generare gli esperimenti, raccogliere i risultati e produrre le visualizzazioni è disponibile nel seguente repository:

[<https://github.com/Di-Simone-dev/AE-MaxFlow>]

Il repository è organizzato come segue:

- **/src**: implementazioni C++ degli algoritmi di massimo flusso (`push_relabel`, `capacity_scaling`, `almost_linear`) e delle utilità comuni (`util`: parsing DIMACS, aritmetica razionale, scalatura delle capacità);
- **graphgenerator.py**: script Python per la generazione delle istanze sintetiche (`layered`, `grid`, Erdős–Rényi DAG) con capacità intere, unitarie, razionali e irrazionali, in formato DIMACS `.max`;
- **/configs**: file di configurazione TOML per la pipeline C++ (`configmain.toml`: flag CLI, percorsi di output, dataset reali) e per la pipeline Python (`config.toml`);
- **main.cpp**: orchestrazione dei benchmark, sia su istanze sintetiche che su reti reali (dataset BVZ–tsukuba e KZ2–venus), con raccolta automatica delle metriche (tempo, flusso, correttezza) in formato CSV;
- **aggregate_and_plot.py**: script per l'aggregazione dei CSV grezzi (media/mediana dei tempi) e la produzione dei grafici di scalabilità utilizzati nel report;
- **/Dati_PAPER**: istanze di test, CSV grezzi e aggregati, tabelle LaTeX e grafici effettivamente utilizzati per i risultati presentati in questo report.

Il repository include inoltre istruzioni dettagliate per la riproduzione degli esperimenti (guide separate per Linux e Windows), comprese le dipendenze software e i parametri utilizzati.

11.1 Riferimenti bibliografici

References

- [1] A. V. Goldberg and R. E. Tarjan, “A new approach to the maximum-flow problem,” *Journal of the ACM*, vol. 35, no. 4, pp. 921–940, 1988.
- [2] H. N. Gabow, “Scaling algorithms for network problems,” *Journal of Computer and System Sciences*, vol. 31, no. 2, pp. 148–168, 1985.
- [3] L. Chen, R. Kyng, Y. P. Liu, R. Peng, M. Probst Gutenberg, and S. Sachdeva, “Almost-linear-time algorithms for maximum flow and minimum-cost flow,” *Communications of the ACM*, vol. 67, no. 1, pp. 82–91, 2024. Originally appeared in FOCS 2022 as “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time”.
- [4] R. A. Howard, *Dynamic Programming and Markov Processes*. MIT Press, 1960.
- [5] A. V. Borup and A. R. Nielsen, “The feasibility of implementing maximum flow and minimum-cost flow in almost-linear time with an IPM,” research project report, IT University of Copenhagen, Dec. 2024. Supervisors: Riko Jacob, Thore Husfeldt. Available at <https://github.com/adbo-ITU/maximum-flow-and-minimum-cost-flow-in-almost-linear-time>.